

A Project report on

**SMART FERTILIZER SPRAYING SYSTEM USING PLANT
DISEASE SEVERITY ANALYSIS**

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

**COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**

By

G. SREEJA	(224G1A33A9)
B. REVATHI	(224G1A3380)
Y. SRAVANI	(224G1A33A4)
Y. TRINESH REDDY	(234G5A3311)

Under the Guidance of

Dr. C. Sasikala M. Tech., Ph.D.
Associate Professor



Department of Computer Science and Engineering
(Artificial Intelligence & Machine Learning)
SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)
Rotarypuram Village, B K Samudram Mandal, Ananthapuramu – 515701
2025-2026

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to JNTUA, Accredited by NAAC with 'A' Grade, Approved by AICTE, New Delhi & Accredited by NBA (CIV, EEE, MEC, ECE & CSE))

Rotarypuram Village, BK Samudram Mandal, Ananthapuramu-515701

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)



Certificate

This is to certify that the project report entitled **SMART FERTILIZER SPRAYING SYSTEM USING PLANT DISEASE SEVERITY ANALYSIS** is the bonafide work carried out by **G. Sreeja** bearing Roll Number **224G1A33A9** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering (Artificial Intelligence & Machine Learning)** during the academic year 2025-2026.

Project Guide

Dr. C. Sasikala M. Tech., Ph.D.
Associate Professor

Head of the Department

Dr. P. Chitralingappa, M. Tech., Ph.D.
Associate Professor
Dept. of CSE(AI&ML)

Date:

Place: Rotarypuram

External Examiner

DECLARATION

I Ms. G. Sreeja bearing reg no: 224G1A33A9 bearing student of **SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY**, Rotarypuram, hereby declare that the dissertation entitled “**SMART FERTILIZER SPRAYING SYSTEM USING PLANT DISEASE SEVERITY ANALYSIS**” embodies the report of our project work carried out by my during IV year Bachelor of Technology under the guidance of **Dr. C. Sasikala** M. Tech, Ph.D., Associate Professor Department of CSE and this work has been submitted for the partial fulfillment of the requirements for the award of the Bachelor of Technology degree.

The results embodied in this project have not been submitted to any other University of Institute for the award of any Degree or Diploma.

Date:

Place:

S. No.	Name of the Student	Roll Number	Signature
1	G. SREEJA	224G1A33A9	
2	B. REVATHI	224G1A3380	
3	Y. SRAVANI	224G1A33A4	
4	Y. TRINESH REDDY	234G5A3311	

Vision & Mission of the SRIT

Vision:

To become a premier Educational Institution in India offering the best teaching and learning environment for our students that will enable them to become complete individuals with professional competency, human touch, ethical values, service motto, and a strong sense of responsibility towards environment and society at large.

Mission:

- Continually enhance the quality of physical infrastructure and human resources to evolve in to a center of excellence in engineering education.
- Provide comprehensive learning experiences that are conducive for the students to acquire professional competences, ethical values, life-long learning abilities and understanding of the technology, environment and society.
- Strengthen industry institute interactions to enable the students work on realistic problems and acquire the ability to face the everchanging requirements of the industry.
- Continually enhance the quality of the relationship between students and faculty which is a key to the development of an exciting and rewarding learning environment in the college.

Vision & Mission of the Department of CSE(AI&ML)

Vision:

To evolve as a leading department by offering best comprehensive teaching and learning practices for students to be self-competent technocrats with professional ethics and social responsibilities.

Mission:

DM 1: Continuous enhancement of the teaching-learning practices to gain profound knowledge in theoretical & practical aspects of computer science applications.

DM 2: Administer training on emerging technologies and motivate the students to inculcate self-learning abilities, ethical values and social consciousness to become competent professionals.

DM 3: Perpetual elevation of Industry-Institute interactions to facilitate the students to work on real-time problems to serve the needs of the society.

Program Educational Objectives (PEOs)

An SRIT graduate in Computer Science & Engineering, after three to four years of graduation will:

PEO 1: Lead a successful professional career in IT / ITES industry / Government organizations with ethical values.

PEO 2: Become competent and responsible computer science professional with good communication skills and leadership qualities to respond and contribute significantly for the benefit of society at large.

PEO 3: Engage in life-long learning, acquiring new and relevant professional competencies / higher academic qualifications.

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned my efforts with success. It is a pleasant aspect that I have now the opportunity to express my gratitude for all of them.

It is with immense pleasure that I would like to express my indebted gratitude to my Guide **Dr. C. Sasikala**, M.Tech., Ph.D., **Associate Professor, Computer Science & Engineering**, who has guided me a lot and encouraged me in every step of the project work. I thank her for the stimulating guidance, constant encouragement and constructive criticism which have made possible to bring out this project work.

I express my deepfelt gratitude to **Mr. B. Vijaya Bjaskar Reddy** M. Tech, CSE (AI & ML), project coordinator for his valuable guidance and unstinting encouragement enabled me to accomplish my project successfully in time

I am very much thankful to **Dr . Chitralingappa**, M. Tech., Ph.D., **Associate Professor& Head of the Department, Computer Science& Engineering (AI&ML)**, for his kind support and for providing necessary facilities to carry out the work.

I wish to convey my special thanks to **Dr. G. Bala Krishna**, Ph.D., **Principal of Srinivasa Ramanujan Institute of Technology (Autonomous)** for giving the required information in doing myr project work. Not to forget, I thank all other faculty and non- teaching staff, and my friends who had directly or indirectly helped and supported me in completing my project in time.

I also express my sincere thanks to the Management for providing excellent facilities.

Finally, I wish to convey my gratitude to my families who fostered all the requirements and facilities that I need.

Project Associate
224G1A33A9

ABSTRACT

The growing need for precision agriculture calls for smart systems that can detect crop diseases and improve chemical use. Traditional fertilizer spraying methods often apply the same amount regardless of disease severity. This results in wasted resources and harm to the environment. To tackle this issue, this paper presents a Smart Fertilizer Spraying System that includes deep learning for classifying tomato disease severity. The system uses a Raspberry Pi with a USB camera and a YOLOv8 model built with the Py torch framework. It analyzes images of tomato fruits and classifies them into three disease severity levels: low, medium, and high. Based on the predicted severity, control signals go to an Arduino microcontroller, which adjusts the fertilizer spraying time using relay-controlled pumps. The robotic platform allows for automated movement and operation. Experimental tests showed a model accuracy of about 75 to 80 percent under controlled conditions. This severity-based spraying method reduces fertilizer waste when compared to traditional uniform spraying and improves crop management precision. The system supports smarter automation in sustainable agriculture.

Key Words—Precision Agriculture, YOLOv8, Deep Learning, Disease Severity Classification, Raspberry Pi, Arduino, Automated Fertilizer Spraying, Smart Farming.

PROJECT CONTRIBUTION TOWARDS SUSTAINABLE DEVELOPMENT GOALS

Project Title:

Smart Fertilizer Spraying System Using Plant Disease Severity Analysis

SDG - Project Mapping:

S.No.	Sustainable Development Goals	Mapped (Yes/No)
1	SDG 1: No Poverty	No
2	SDG 2: Zero Hunger	Yes
3	SDG 3: Good Health and Well-Being	Yes
4	SDG 4: Quality Education	Yes
5	SDG 5: Gender Equality	Yes
6	SDG 6: Clean Water and Sanitation	No
7	SDG 7: Affordable and Clean Energy	No
8	SDG 8: Decent Work and Economic Growth	No
9	SDG 9: Industry Innovation and Infrastructure	Yes
10	SDG 10: Reduced Inequalities	Yes
11	SDG 11: Sustainable Cities and Communities	Yes
12	SDG 12: Responsible Consumption and Production	Yes
13	SDG 13: Climate Action	Yes
14	SDG 14: Life Below Water	No
15	SDG 15: Life on Land	No
16	SDG 16: Peace Justice and Strong Institutions	Yes
17	SDG 17: Partnerships for the Goals	No

Explanation:

- ✓ The project enhances sustainable agriculture by detecting plant diseases and applying fertilizer accurately (SDG 2 & SDG 3).
- ✓ It supports smart and innovative farming with efficient resource utilization and two key technologies: Raspberry Pi and Arduino (SDG 9).
- ✓ Solar-assisted power and automated fertilizer spraying reduce environmental impact and promote eco-friendly farming practices (SDG 12 & SDG 13).

Expected Impact:

Parameter	Impact
Fertilizer Usage	Reduced by 20-40%
Crop Yield	Increased by 15-25%
Labour Effort	Minimised for Monitoring and Spraying

CONTENTS

	Page No
List of Figures	IX-X
List of Tables	XI
List of Abbreviations	XII
Chapter 1 Introduction	1–5
1.1 Internet of Things	1
1.2 Objectives	4
1.3 Problem Statement	5
Chapter 2 Literature Survey	6–8
Chapter 3 Design	9–23
3.1 Existing System	9
3.1.1 System Architecture	9
3.1.2 Hardware	10
3.1.3 Disadvantages	10
3.2 Proposed System	10
3.2.1 System Architecture	11
3.3 Software Design	12
3.3.1 Programming Languages	13
3.3.1.1 Embedded C	13
3.3.1.2 Python	14
3.3.1.3 OpenCV	14
3.3.1.4 YOLOv8 Framework	15
3.3.1.5 PyTorch	16
3.3.2 Arduino IDE	17

3.3.3 Raspbian OS	18
3.4 Functional Requirements	19
3.5 Non-Functional Requirements	19
3.5.1 Portability	19
3.5.2 Speed	19
3.5.3 Reliability	19
3.6 Methodology	20
3.7 Cost Estimation	21
3.8 Time Estimation	21
3.9 System Architecture	22
3.10 Circuit Architecture	23
 Chapter 4 Implementation	 24–53
4.1 Software Implementation	24
4.1.1 Dataset Preparation	24
4.1.2 YOLOv8 Model Training	25
4.1.3 Real-Time Image Capture	26
4.1.4 Disease Detection using YOLOv8	27
4.1.5 Serial Communication with Arduino	28
4.1.6 Fertilizer Spraying Mechanism	30
4.1.7 Installation of Arduino IDE	30
4.1.8 Hardware Code	35
4.1.9 Python and YOLOv8 Environment Setup	39
4.1.10 OpenCV Installation and Image Processing	40
4.1.11 Raspberry Pi Configuration	41
4.1.12 System Workflow Implementation	42
4.1.13 Advanced IP Scanner	43
4.1.14 RealVNC Viewer	45

4.2 Hardware Implementation	46
4.2.1 Raspberry Pi Implementation	47
4.2.2 Arduino Uno Implementation	48
4.2.3 Web cam	49
4.2.4 Motor Driver	50
4.2.5 Relay Module	51
4.2.6 Water pump	52
4.2.7 Battery	52
4.2.8 Bluetooth Module	53
Chapter 5 Testing	54–58
5.1 Testing Approach	54
5.2 Features to be Tested	54
5.3 Testing Tools and Environment	56
5.4 Testing Environment	57
5.5 Testcases	57
5.5.1 NonFunctional Requirements	58
5.5.2 Portability	58
5.5.3 Speed	58
Chapter 6 Results	59-60
Chapter 7 Conclusion	62-63
References	64
Patent Publication Status	65
Internship Certificates	66-69
Project CO-OP Mapping	70
Details of Project Team	71

List of Figures

Fig. No.	Description	Page No.
1.1	Overview of Embedded System	2
1.2	Block diagram of Embedded System	3
3.1	Block diagram	12
3.2	YOLOv8 Working Diagram	16
3.3	Arduino IDE	17
3.4	Raspbian OS	18
3.5	Circuit Diagram of Proposed System	23
4.1	Tomato Fruit Affected by Blossom End Rot Disease	25
4.2	YOLO Model Workflow For Tomato Disease Detection	26
4.3	Real time Image Capture Using Webcam	27
4.4	Serial Communication	29
4.5	YOLOv8 based Tomato Disease Detection Output	40
4.6	Block Diagram of Autonomous Plant Disease Detection	40
4.7	Detecting raspberry Pi IP address	44
4.8	Remote access to raspberry Pi using Real VNC Viewer	46
4.9	Raspberry Pi	48
4.10	Arduino Uno	49
4.11	Webcam	50
4.12	Dc Motor	51
4.13	Relay Module	51

Fig. No.	Description	Page No.
4.14	DC Water Pump	52
4.15	Battery	53
4.16	HC-05 Bluetooth Module	53
6.1	Training and Validation Accuracy	60
6.2	Confusion Matrix of YOLOv8	61
6.3	Precision and Recall Comparison	61

List of Tables

Table No.	Table Name	Page No.
3.1	Cost Estimation	19
3.2	Time Estimation	20
5.1	Test cases	57

List of Abbreviations

Abbreviation	Full Form
YOLOv8	You Only Look Once Version 8
RPi	Raspberry Pi
MCU	Microcontroller Unit
L293D	Motor Driver Integrated Circuit
USB Cam	Universal Serial Bus Camera
DC Motor	Direct Current Motor
CNN	Convolutional Neural Network
ML	Machine Learning
CV	Computer Vision
GPU	Graphics Processing Unit

CHAPTER 1

INTRODUCTION

1.1 Internet of Things

The Internet of Things (IoT) refers to a network of interconnected physical devices that communicate and exchange data through the internet. These devices are embedded with sensors, software, and communication technologies that enable them to collect, process, and transmit data without requiring constant human intervention. IoT plays a crucial role in automation and intelligent decision-making across various domains such as healthcare, smart homes, transportation, manufacturing, and agriculture.

In agriculture, IoT technologies enable farmers to monitor crop conditions, soil moisture, temperature, and plant health in real time. By integrating IoT with advanced technologies such as artificial intelligence (AI), machine learning, and computer vision, farmers can automate agricultural processes and make data-driven decisions. These smart farming techniques improve productivity, reduce resource wastage, and support sustainable agricultural practices.

One of the major challenges in agriculture is the early detection of plant diseases and the efficient application of fertilizers or pesticides. Traditional farming methods rely heavily on manual observation, which is time-consuming and prone to human error. Additionally, conventional fertilizer spraying methods often apply the same amount of chemicals regardless of the severity of plant disease. This leads to excessive chemical usage, increased production costs, and environmental pollution.

Recent advancements in deep learning and computer vision provide effective solutions for automated plant disease detection. Object detection algorithms such as YOLO (You Only Look Once) can identify and classify plant diseases from images with high speed and accuracy. When integrated with IoT-based embedded systems such as Raspberry Pi and Arduino, these technologies can enable automated agricultural systems that not only detect diseases but also perform appropriate actions such as targeted fertilizer spraying.

The proposed system utilizes a Raspberry Pi with a USB camera to capture images of tomato plants and analyze them using a YOLOv8 deep learning model. Based on the detected disease severity level, the system automatically controls fertilizer spraying through an Arduino-based actuator mechanism. This approach helps optimize fertilizer usage, reduce human intervention, and promote precision agriculture.

Embedded into the hardware. One of its most important characteristics of these systems is, it gives the o/p within the time limits. Embedded systems support to make the work more perfect and convenient. So, we frequently use embedded systems in simple and complex devices too. The applications of embedded systems mainly involve in our real life for several devices like microwave, calculators, TV remote control, home security and neighborhood traffic control systems, etc. Overview of Embedded System is shown in Fig.1.1

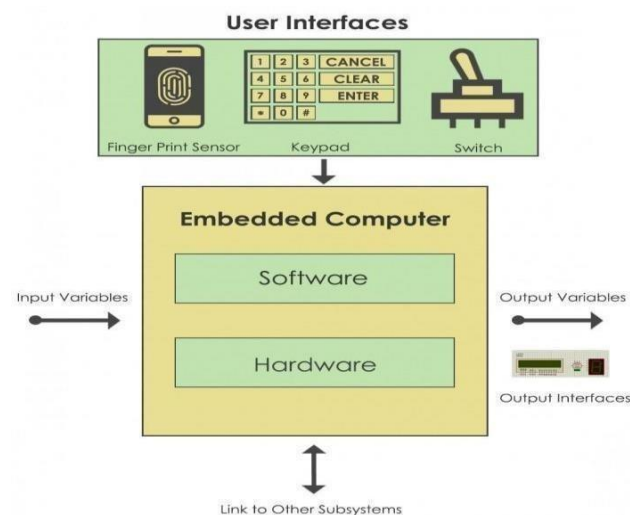


Fig.1.1: Overview of Embedded System

Embedded System

Embedded system includes mainly two sections:

- 1 **Hardware**
- 2 **Software**

The block diagram of Embedded system is shown below in Fig.1.2

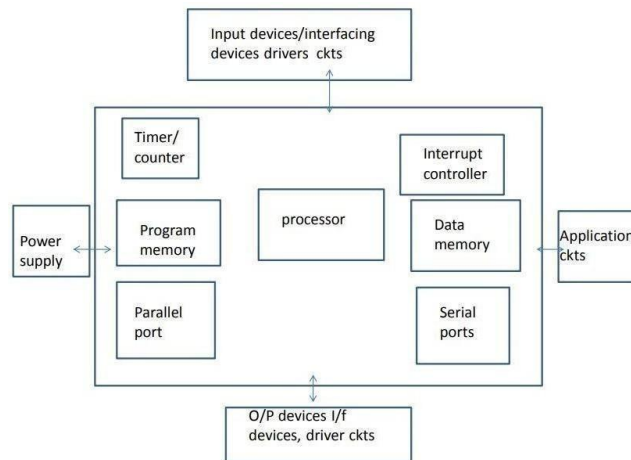


Fig.1.2: Block Diagram of Embedded System

Embedded System Hardware

As with any electronic system, an embedded system requires a hardware platform on which it performs the operation. Embedded system hardware is built with a microprocessor or microcontroller. The embedded system hardware has elements like input output (I/O) interfaces, user interface, memory and the display. Usually, an embedded system consists of:

- Power Supply
- Processor
- Memory
- Timers
- Serial communication ports
- Output/Output circuits
- System application specific circuits

Embedded systems use different processors for its desired operation. Some of the processors used are

1. Microprocessor
2. Microcontroller
3. Digital signal processor

Embedded System Software

The embedded system software is written to perform a specific function. It is typically written in a high-level format and then compiled down to provide code that can be lodged within a non-volatile memory within the hardware. An embedded system software is designed to keep given the three limits:

- Availability of system memory
- Availability of processor's speed
- When the system runs continuously, there is a need to limit power dissipation for events like stop, run, and wake up.

1.2 Objectives

The main objectives of the proposed system are:

- To develop an AI-based system for detecting and classifying tomato disease severity using deep learning.
- To implement a **YOLOv8** model for real-time image analysis and severity detection.
- To design an IoT-based system using **Raspberry Pi** and **Arduino** for automated fertilizer spraying.
- To optimize fertilizer usage based on detected disease severity levels.
- To reduce manual labor and improve precision agriculture through automation.

- To support sustainable farming by minimizing excessive fertilizer usage and environmental impact.

1.3 Problem Statement

Agriculture plays a vital role in global food production and economic development. However, crop diseases significantly affect agricultural productivity and lead to major economic losses for farmers. Early detection and proper treatment of plant diseases are essential for maintaining healthy crop growth and maximizing yield.

Traditional disease detection methods depend on manual inspection by farmers or agricultural experts. These methods are often slow, labor-intensive, and may lead to incorrect diagnosis due to human error. Moreover, conventional fertilizer spraying techniques apply a uniform amount of chemicals across the entire field without considering the actual severity of the disease. This results in unnecessary chemical usage, increased operational costs, and harmful environmental effects.

Another major challenge is the lack of automated systems that can simultaneously detect disease severity and perform targeted fertilizer application. Many existing research works focus only on disease detection but still require manual intervention for spraying.

Therefore, there is a need for an intelligent agricultural system that can automatically detect plant disease severity and control fertilizer spraying accordingly. The proposed **Smart Fertilizer Spraying System** aims to address this problem by integrating **deep learning- based disease severity detection with an IoT-enabled automated spraying mechanism**. This system helps improve crop management, reduce fertilizer wastage, and support sustainable precision agriculture.

CHAPTER-2

LITERATURE SURVEY

[1].This paper presents a plant leaf disease detection system using image processing techniques. The approach focuses on identifying disease symptoms from leaf images through preprocessing, segmentation, and feature extraction. The study demonstrates that digital image processing can effectively assist farmers in identifying plant diseases early and improving crop productivity.

[2].The study introduces a plant monitoring system using image processing, Raspberry Pi, and IoT technologies. The system captures plant images and processes them to detect diseases, while IoT enables remote monitoring of plant health conditions. The integration of embedded hardware and image analysis provides a cost-effective and automated agricultural monitoring solution.

[3].This research focuses on identifying cotton leaf diseases using Raspberry Pi and image processing methods. The system captures leaf images through a camera module and processes them to detect disease patterns. The use of a low-cost embedded platform allows real-time disease detection and supports precision agriculture applications.

[4].The paper proposes a special method for detecting grape leaf diseases using image processing techniques. The system analyzes leaf images to identify disease-affected regions through segmentation and classification methods. The study demonstrates improved accuracy in identifying grape diseases, helping farmers take timely preventive measures.

[5].This work presents an OpenCV-based system for detecting diseases in mango leaves. The approach uses image preprocessing, feature extraction, and classification to identify disease symptoms. The use of computer vision techniques provides a practical method for early detection of plant diseases in mango crops.

[6].This research introduces a deep learning approach for image-based plant disease detection. Using convolutional neural networks (CNNs), the model automatically learns features from plant leaf images and classifies different diseases with high accuracy. The study highlights the effectiveness of deep learning in agricultural diagnostics.

[7].The paper presents the YOLO (You Only Look Once) object detection algorithm, a real-time deep learning model for detecting objects in images. YOLO processes images in a single neural network pass, enabling fast and accurate detection. This technique has significant applications in agricultural systems for identifying plant diseases and affected regions in real time.

[8].This research introduces the SSD (Single Shot MultiBox Detector), a deep learning model designed for fast object detection. SSD detects objects at multiple scales using convolutional neural networks, providing high accuracy and speed. The method can be applied to agricultural image analysis for identifying diseased plant areas efficiently.

[9].This paper proposes the AlexNet deep convolutional neural network for large-scale image classification. The model significantly improved image recognition accuracy by using deep CNN architectures and GPU training. The work laid the foundation for modern deep learning approaches used in plant disease detection systems.

[10].This study presents the development of a smart pesticide spraying robot that uses image processing and machine learning techniques. The robot detects diseased plants and sprays pesticides automatically only on affected areas. This targeted approach reduces chemical usage and promotes sustainable farming practices.

[11] This paper provides a comprehensive survey of deep learning methods used for crop disease detection and diagnosis. It reviews various CNN architectures, datasets, and classification techniques applied in agriculture. The study highlights the potential of deep learning for improving accuracy and efficiency in plant disease identification.

[12] .This research reviews machine learning applications in precision agriculture. It discusses the use of AI techniques for crop monitoring, disease detection, yield prediction, and resource optimization. The paper emphasizes how machine learning can enhance agricultural productivity and support smart farming system

CHAPTER 3

DESIGN

3.1 Existing System

Traditional agricultural disease monitoring systems mainly rely on manual observation and conventional farming practices. Farmers inspect crops visually to identify symptoms of diseases, pest attacks, or nutrient deficiencies. Based on their observations, fertilizers or pesticides are applied using manual sprayers or tractor-mounted equipment.

Although this approach has been widely used for many years, it has several limitations. Manual inspection is time-consuming and requires significant human effort. Moreover, the accuracy of disease detection depends heavily on the experience of farmers. Misinterpretation of disease symptoms can lead to improper fertilizer application.

Several research works have proposed image-processing-based plant disease detection systems. These systems capture images of plant leaves and apply algorithms to identify disease patterns. While these solutions improve disease detection accuracy, most of them only provide advisory recommendations. Farmers still need to manually apply fertilizers or pesticides after receiving the analysis results.

In addition, many existing systems focus only on identifying the presence of disease rather than evaluating its severity. As a result, fertilizer spraying is often performed uniformly across crops without considering the extent of infection. This leads to inefficient resource usage, increased chemical wastage, and environmental damage.

3.1.1 System Architecture

The architecture of traditional plant monitoring systems typically includes three stages:

1. Image acquisition
2. Disease detection
3. Advisory recommendation

The images of plant leaves are captured using cameras and analyzed using image processing algorithms. The system then provides recommendations to farmers regarding disease treatment. However, the actual spraying of fertilizers is still performed manually.

3.1.2 Hardware

Existing systems usually use limited hardware components such as cameras, simple microcontrollers, and manual spraying tools. These systems do not include integrated robotic platforms or automated spraying mechanisms.

3.1.3 Disadvantages

The disadvantages of the existing system include:

- Manual crop inspection is time consuming.
- Disease detection accuracy depends on farmer expertise.
- Lack of automated fertilizer spraying mechanisms.
- Uniform fertilizer spraying leads to resource wastage.
- Increased labor requirements.
- Environmental damage due to excessive chemical usage.
- These limitations highlight the need for an automated precision agriculture system.

3.2 Proposed System

The proposed Smart Fertilizer Spraying System integrates artificial intelligence, image processing, and robotic automation to improve agricultural productivity. The system detects disease severity in crops and automatically controls fertilizer spraying based on the detected severity level.

A USB webcam captures real-time images of tomato crops. These images are processed by a Raspberry Pi that runs a trained YOLOv8 deep learning model. The model analyzes the images and classifies the disease severity into three categories: **low, medium, and high**.

After detecting the severity level, the Raspberry Pi sends the classification result to an

Arduino Uno microcontroller through serial communication. The Arduino then activates relay modules connected to fertilizer pumps. Each severity level corresponds to a predefined spraying duration, ensuring accurate fertilizer application.

The system also includes a robotic platform driven by DC motors. The motors are controlled using a motor driver, allowing the robot to move across the agricultural field.

This automated approach reduces human intervention and improves precision fertilizer application.

3.2.1 System Architecture

The overall architecture of the proposed system is illustrated in Figure 3.1 (Block Diagram).

The system consists of four main modules:

Image Acquisition Module

A USB webcam captures images of tomato crops in real time. These images are transmitted to the Raspberry Pi for further processing.

Disease Detection Module

The Raspberry Pi processes the captured images using a trained YOLOv8 deep learning model. The model identifies disease severity and classifies it into low, medium, or high categories.

Control and Actuation Module

After classification, the Raspberry Pi sends commands to the Arduino Uno. The Arduino controls relay modules that activate fertilizer pumps and motor drivers that control robot movement.

Power Management Module

The system is powered by a 12V battery supported by a solar panel. This ensures continuous system operation in agricultural environments. The block diagram is shown below in Fig 3.1

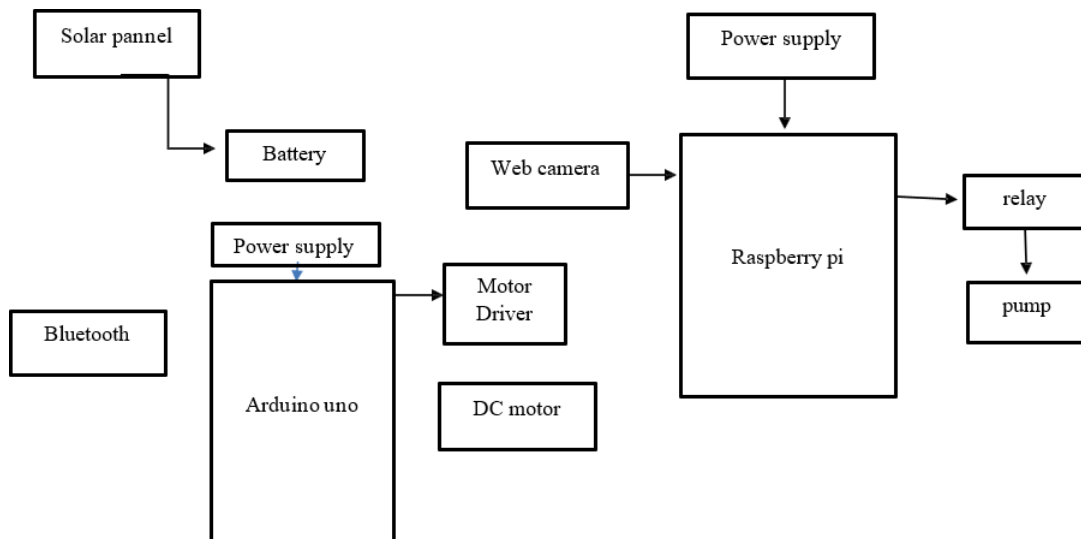


Fig.3.1: Block Diagram

3.3 Software Design

The software components used in the system include:

- Programming languages
- Open CV
- YOLOv8 Framework

- PyTorch Deep Learning Library
- Arduino IDE
- Raspbian Operating System

3.3.1 Programming languages

3.3.1.1 EMBEDDED C

Embedded C makes use of KEIL IDE programming. The framework program written in embedded C can be placed away in Microcontroller. The accompanying is a portion of the actual motives behind composing applications in C as opposed to get collectively. It is much less disturbing and much less tedious to write down in C than amassing. C is less traumatic to trade and refresh. You can utilize code available in capacity libraries. C code is compact to different microcontrollers with subsequent to 0 alteration. Genuine, installed C programming need nonstandard expansions to the C driver with a view to bolster charming components, as an example, settled point range catching, numerous unmistakable reminiscence banks, and fundamental I/O operations.

In 2008, the C Standards Committee prolonged the C data to deal with these problems via giving a normal well known to all executions to purchaser to contains numerous additives not handy in standard C, for example, settled factor wide variety catching, named address spaces, and vital I/O equipment tending to.

Installed C utilize the greater part of the grammar and semantics of wellknown C, e.G., number one() paintings, variable definition, facts type statement, contingent proclamations (if, switch. Case), circles (even as, for), capacities, exhibits and strings, structures and union, piece operations, macros, unions, and so on.

3.3.1.2 Python

Python is the primary programming language used in the development of the proposed system.

It is widely used in artificial intelligence and machine learning applications due to its simplicity, flexibility, and extensive library support.

In this project, Python is used to:

- Load and run the trained YOLOv8 model
- Capture real-time images from the webcam
- Perform image processing and object detection
- Communicate with the Arduino microcontroller through serial communication
- Python libraries provide efficient tools for handling image processing and machine learning tasks. Its compatibility with Raspberry Pi also makes it suitable for embedded AI applications.

3.3.1.3 Open CV

OpenCV (Open Source Computer Vision Library) is used for image acquisition and image processing tasks. It enables the system to capture frames from the webcam and prepare them for analysis by the YOLOv8 model.

In the proposed system, OpenCV performs the following functions:

- Capturing images from the USB webcam
- Frame preprocessing
- Displaying detection results
- Passing images to the deep learning model

OpenCV provides optimized algorithms that allow efficient image processing on embedded devices such as Raspberry Pi.

3.3.1.4 Ultralytics YOLOv8 Framework

The YOLOv8 framework is used for real-time disease detection and severity classification. YOLO (You Only Look Once) is a state-of-the-art object detection algorithm designed for high-speed and accurate detection.

The YOLOv8 model is trained using labeled images of tomato crops categorized into three severity classes:

- Low severity
- Medium severity
- High severity

During real-time operation, the Raspberry Pi processes webcam images using the trained YOLOv8 model. The model predicts the severity category and sends the corresponding command to the Arduino. The working of YOLO is shown below in Fig.3.2

Key advantages of YOLOv8 include:

- Fast real-time detection
- High detection accuracy
- Efficient performance on embedded systems such as Raspberry Pi

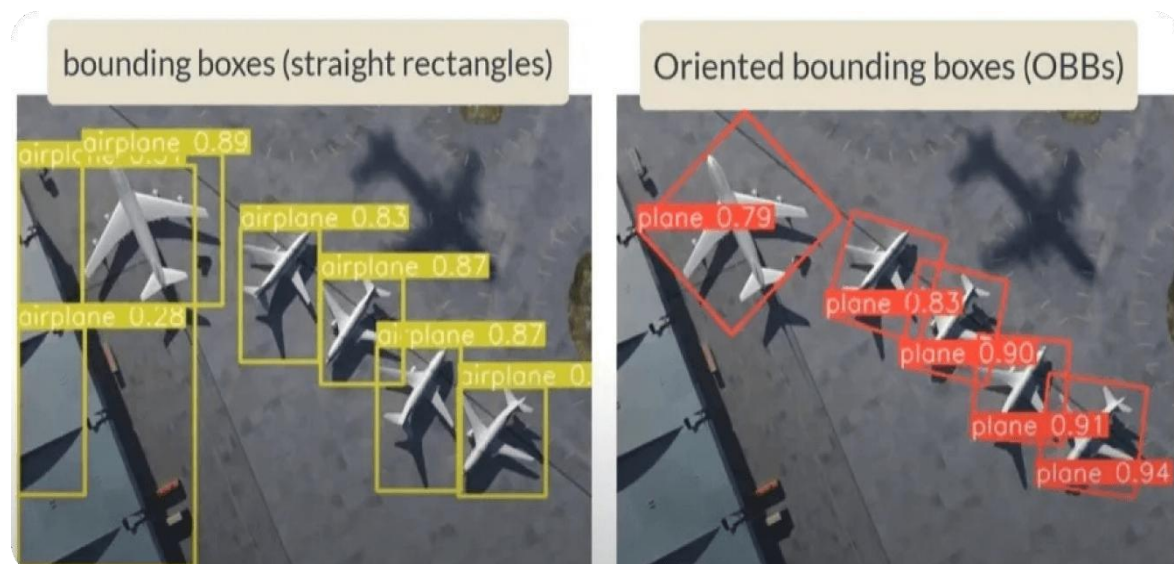


Fig.3.2: YOLOv8 Working Diagram

3.3.1.5 PyTorch Deep Learning Library

PyTorch is the deep learning framework used to train the YOLOv8 model. It provides powerful tools for building and training neural networks.

PyTorch offers several advantages such as:

- Dynamic computational graphs
- Efficient GPU and CPU computation
- Easy integration with Python
- Extensive support for deep learning research

In this project, PyTorch is used to train the disease detection model and generate the trained model weights file (best.pt) that is deployed on the Raspberry Pi for real-time inference.

3.3.1 Arduino IDE

Arduino IDE is used to develop and upload programs to the Arduino Uno microcontroller. It provides a simple programming environment for controlling hardware components such as motors, relay modules, and water pumps.

In this project, the Arduino program performs the following tasks:

- Receive commands from the Raspberry Pi
- Control relay modules connected to fertilizer pumps
- Activate pumps for predefined durations

- Control the movement of DC motors

The Arduino IDE simplifies hardware programming and enables seamless integration with the Raspberry Pi is shown in Fig 3.3

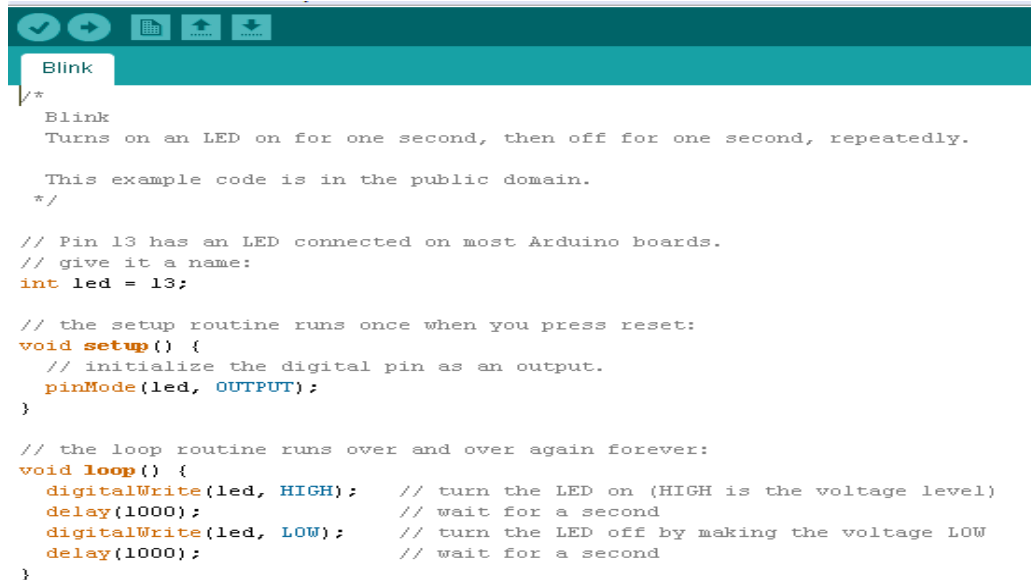


Fig.3.3: Arduino IDE

3.3.2 Raspbian Operating System

Raspbian OS is the operating system installed on the Raspberry Pi. It is a Linux-based operating system specifically optimized for Raspberry Pi hardware. Interface for Raspbian OS is shown in Fig3.4

The Raspbian environment allows the system to:

- Run Python programs
- Execute the YOLOv8 detection model
- Communicate with the Arduino through serial ports
- Manage connected hardware devices

Its lightweight design ensures efficient operation of the AI-based agricultural monitoring system.

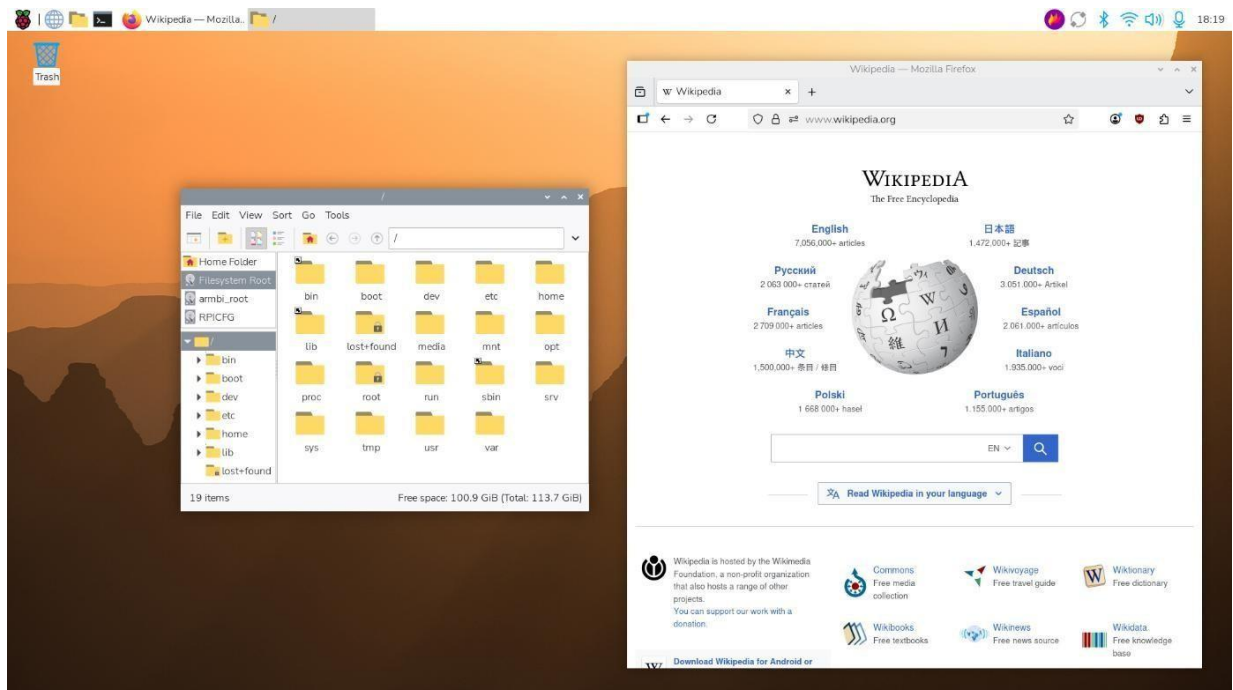


Fig.3.4: Raspbian OS

3.4 Functional Requirements

The system must perform the following functions:

- Capture images using a webcam
- Detect disease severity using deep learning
- Classify severity into low, medium, or high
- Send commands to Arduino
- Control fertilizer spraying automatically

3.5 Non-Functional Requirements

3.5.1 Portability

The system should be portable and suitable for field deployment.

3.5.2 Speed

The system should perform disease detection in real time.

3.5.3 Reliability

The system should operate continuously without system failure.

3.6 Methodology

The methodology of the proposed Smart Fertilizer Spraying System involves several stages including data collection, model training, real-time detection, and automated fertilizer spraying. The objective of the methodology is to accurately detect the severity of tomato diseases and apply fertilizer based on the detected severity level.

Initially, a dataset containing images of tomato crops was prepared. The dataset consisted of images categorized into three severity levels: low, medium, and high. These images were manually labeled based on the percentage of infection visible on the tomato surface. The labeled dataset was then divided into training, validation, and testing sets to train and evaluate the deep learning model.

A YOLOv8 deep learning model was used for disease severity detection. The model was trained

using the prepared dataset with multiple training epochs to ensure that it learns the visual

characteristics of different severity levels. After training, the model weights were saved and deployed on the Raspberry Pi for real-time inference.

During real-time operation, a USB webcam captures images of tomato crops. These images are processed by the Raspberry Pi, which runs the trained YOLOv8 model. The model analyzes each captured frame and classifies the severity of the disease into one of the predefined categories: low, medium, or high.

After detecting the severity level, the Raspberry Pi sends the classification result to the Arduino Uno microcontroller through serial communication. The Arduino receives the severity command and activates the corresponding fertilizer pump through relay modules.

Each severity level is associated with a specific fertilizer spraying duration. For example, low severity activates the pump for 2 seconds, medium severity activates it for 4 seconds, and high severity activates it for 6 seconds. This severity-based spraying mechanism ensures efficient fertilizer utilization.

The system also includes DC motors controlled by a motor driver to enable robotic movement across the agricultural field. The entire system operates using a battery supported by a solar panel to ensure energy-efficient operation.

3.7 Cost Estimation

Cost estimation is an important aspect of system design to determine the financial feasibility of the proposed solution. The Smart Fertilizer Spraying System is designed using cost-effective components that are easily available in the market.

The estimated cost of the hardware components used in the system is shown below in table 3.1

Table 3.1: Cost Estimation

Cost Estimation		
Component	Quantity	Estimated Cost (INR)
Raspberry Pi	1	4500
Arduino Uno	1	600
USB Webcam	1	800
Motor Driver	1	250
DC Motors	2	600
Relay Module	3	300
Water Pump	3	750
Bluetooth Module	1	350
Battery	1	1200
Connecting Wires	-	200
Total Estimated Cost ≈ 10,550 INR		

3.8 Time Estimation

The development of the Smart Fertilizer Spraying System with Plant Disease Severity Analysis was carried out through several stages, including dataset preparation, model training, hardware integration, and system testing. Each stage required a specific time duration to ensure proper implementation and system reliability. Time estimation is shown in below table 3.2

Table 3.2: Time estimation

3.9 Time Estimation:

S.No	Activity	Duration
1	Problem identification and requirement analysis	1 Week
2	Literature survey and research study	2 Weeks
3	Dataset collection and labeling	1 Week
4	YOLOv8 model training and validation	2 Weeks
5	Hardware assembly and circuit implementation	2 Weeks
6	Integration of Raspberry Pi and Arduino communication	1 Week
7	System testing and performance evaluation	1 Week
8	Documentation and final report preparation	1 Week
Total Project Duration ≈ 11 Weeks		

3.9 System Architecture

The software architecture of the proposed system defines how different software components interact to perform disease detection and fertilizer spraying operations.

The system software consists of the following modules: Image Acquisition Module

The image acquisition module captures real-time images of tomato crops

using a USB webcam connected to the Raspberry Pi. The captured images are transferred to the processing module for analysis.

Disease Detection Module

This module performs image processing and disease severity detection using the YOLOv8 deep learning model. The model analyzes each image and identifies the severity of the disease based on the trained dataset.

Communication Module

The communication module is responsible for transferring data between the Raspberry Pi and the Arduino Uno microcontroller. Serial communication is used to send severity commands from the Raspberry Pi to the Arduino.

Control Module

The control module is implemented on the Arduino microcontroller. It receives severity commands and activates relay modules connected to fertilizer pumps. The Arduino also controls DC motors for robot movement.

Actuation Module

The actuation module includes relay modules, water pumps, and DC motors. Based on the commands received from the Arduino, the pumps spray fertilizer for predefined durations.

The modular design of the software architecture ensures easy system maintenance, scalability, and future upgrades.

3.10 Circuit Architecture

The circuit architecture of the proposed system is shown in **Figure 3.5 (Circuit Diagram)**.

The Raspberry Pi is connected to the webcam for image acquisition and communicates with the Arduino Uno through serial communication. The Arduino is connected to relay modules, motor drivers, and water pumps.

The relay modules control the activation of fertilizer pumps, while the motor driver controls the movement of DC motors. The battery and solar panel provide power to the entire system.

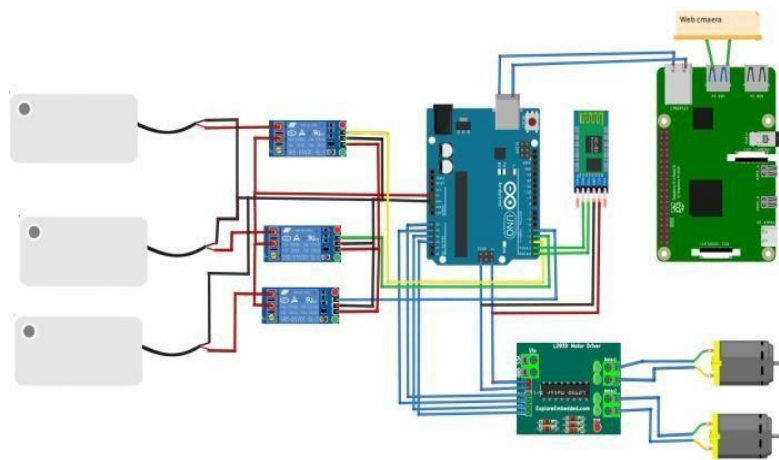


Fig.3.5: Circuit diagram of proposed system

CHAPTER 4

IMPLEMENTATION

4.1 Software implementation

4.1.1 Dataset Preparation

Dataset preparation is the first stage in implementing the Smart Fertilizer Spraying System. A dataset containing images of tomato fruits affected by different diseases is collected and organized for training the deep learning model.

The dataset used in this project contains four classes of tomato conditions:

- Blossom End Rot
- Bacterial Disease
- Cracking
- Normal

The dataset is divided into three parts:

- Training dataset
- Validation dataset
- Testing dataset

The training dataset is used to teach the deep learning model to recognize patterns in diseased tomatoes. The validation dataset is used during training to monitor the model's performance and prevent overfitting, while the testing dataset is used to evaluate the final model after training. The disease affected fruit is shown below in Fig 4.1

Each image in the dataset is annotated with labels that indicate the type of disease present in the tomato fruit. These annotations help the YOLOv8 model learn to identify disease regions and classify them correctly. Proper dataset preparation is essential for achieving accurate disease detection in the system.

Importing the dataset

```
from ultralytics import YOLO
import cv2
import serial
import time
```



Fig.4.1: Tomato fruit affected by Blossom End Rot Disease

4.1.2 YOLOv8 Model Training

The YOLOv8 model is trained using the prepared tomato disease dataset. YOLO (You Only Look Once) is a deep learning-based object detection algorithm used to detect and classify objects in images in real time.

During the training process, the YOLOv8 model analyzes a large number of images from the dataset and learns the visual patterns associated with each disease type. The training process involves multiple iterations called epochs, where in each epoch the model gradually improves its ability to detect diseases more accurately. A pretrained YOLOv8 model is initially loaded before training begins, and it is then fine-tuned using the tomato disease dataset. This transfer learning approach significantly reduces training time and improves detection accuracy.

After the training process is completed, the model generates a trained weight file called best.pt. This file contains the learned parameters of the trained model.

The trained model is then deployed on the Raspberry Pi, where it is used to perform real-time disease detection in tomato plants. The workflow for tomato disease detection is shown below in Fig 4.2

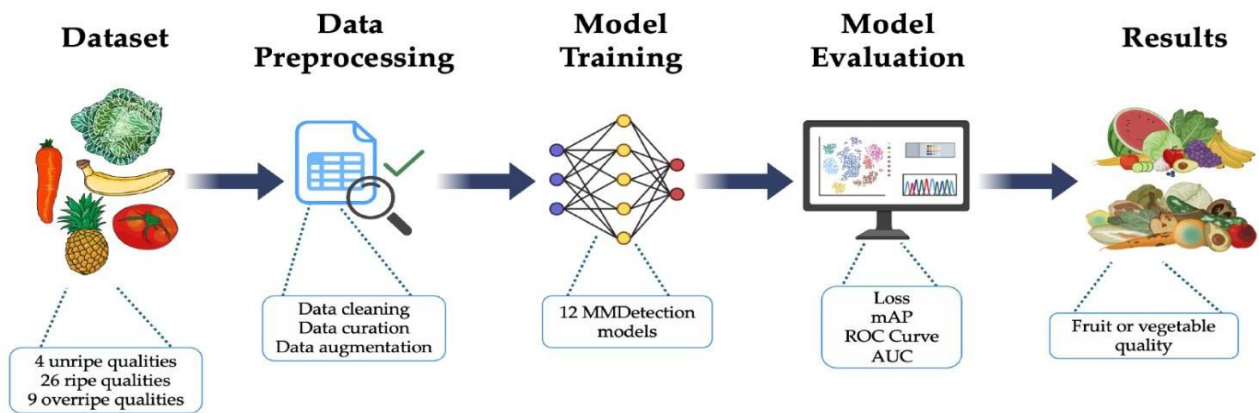


Fig.4.2: YOLO model workflow for tomato disease detection

```
# Load YOLO model: model = YOLO("/home/pi/Desktop/tamota disease/student
dataset/runs/detect/train/weights/best.pt")
```

4.1.3 Real-Time Image Capture

Real-Time Image Capture

Real-time image capture is performed using a USB webcam connected to the Raspberry Pi. The webcam continuously captures images of tomato plants as the robotic platform moves through the agricultural field.

The captured images are processed using the OpenCV library, which enables the Raspberry Pi to capture frames from the webcam and convert them into digital images. These digital images can then be analyzed by the deep learning model.

The real-time monitoring system allows the robot to inspect plants continuously and detect disease symptoms without manual intervention. The captured frames serve as input for the YOLOv8 disease detection model. Real time image capture is shown below in Fig 4.3



Fig.4.3: Real-time image capture using webcam

4.1.4 Disease Detection using YOLOv8:

Once the images are captured, they are analyzed using the trained YOLOv8 model. The model processes each image to detect disease regions present in the tomato fruit.

The YOLOv8 model generates bounding boxes around the detected disease areas and assigns a label to each region based on the type of disease. Along with the label, the model also provides a confidence score, which indicates how certain the model is about its prediction.

For example, the model may detect Blossom End Rot with a confidence score of 0.91. This information helps the system determine both the type and severity of the disease present in the plant.

The detection results are displayed on the screen, allowing users to visually verify the identified disease regions and ensure the accuracy of the system.

YOLO Detection

```
results = model(frame, conf=0.30)
annotated_frame = results[0].plot()

if len(results[0].boxes) > 0:
    box = results[0].boxes[0]
    cls = int(box.cls)
    detected_label = model.names[cls]

cv2.imshow("Tomato Disease Detection", annotated_frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break
cap.release()
cv2.destroyAllWindows()
print("Camera OFF")
```

4.1.5 Serial Communication with Arduino:

Serial Communication with Arduino

After detecting the disease type, the Raspberry Pi sends the detected label to the Arduino microcontroller. The communication between the Raspberry Pi and Arduino is established through serial communication using the USB interface.

The Python program running on the Raspberry Pi sends a text command containing the detected disease label. For example, if the detected disease is Blossom End Rot, the Raspberry Pi transmits the corresponding label to the Arduino.

The Arduino reads the received command and processes it to determine the required action for fertilizer spraying. Based on the received label, the Arduino controls the appropriate hardware components connected to the system.

This communication enables the artificial intelligence system on the Raspberry Pi to effectively interact with and control the hardware components through the Arduino. Serial communication with Arduino and raspberry pi is shown in below Fig 4.4

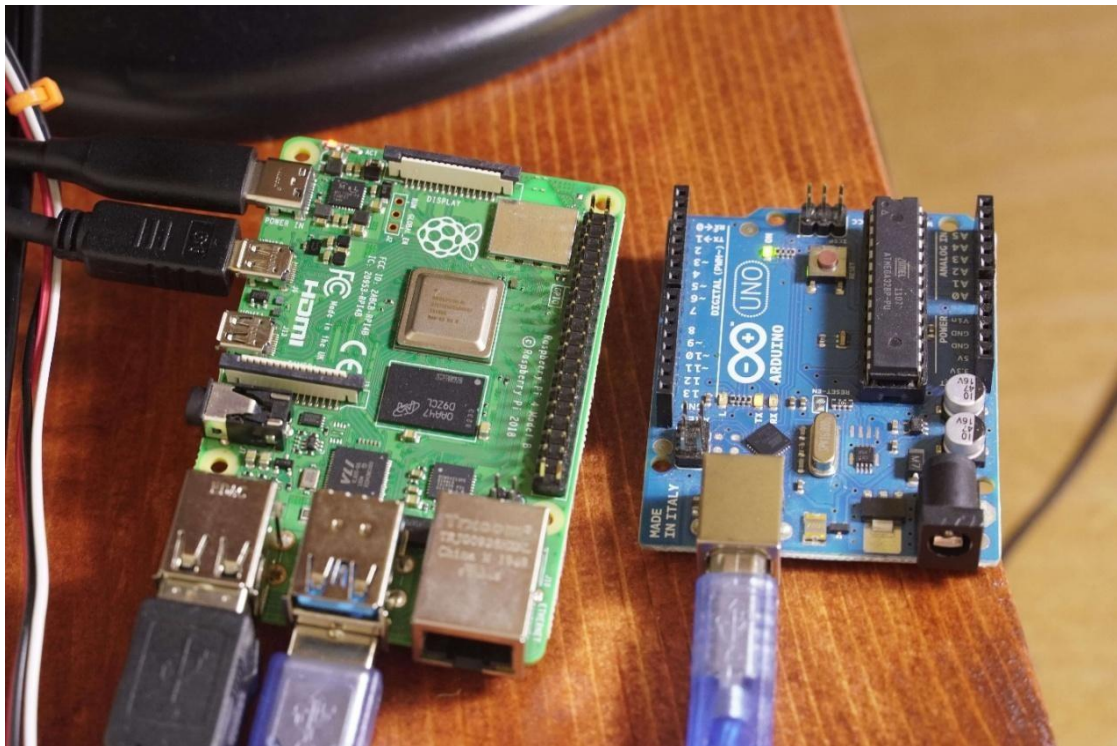


Fig.4.4 Serial Communication

#Serial connection:

```
arduino = serial.Serial('/dev/ttyUSB0', 9600, timeout=1)
```

```
time.sleep(2)
```

```
arduino.reset_input_buffer()
```

#Send detected disease to Arduino:

```
if detected_label:
```

```
    arduino.write((detected_label + "\n").encode())
```

```
    print("Sent to Arduino:", detected_label)
```

```
else:
```

```
    print("No disease detected")
```

```
print("Waiting for next ON signal...")
```

4.1.6 Fertilizer Spraying Mechanism:

Fertilizer Spraying Mechanism

The fertilizer spraying mechanism consists of the following components:

- Relay modules
- Water pumps
- Spray nozzles

The Arduino controls the relay modules based on commands received from the Raspberry Pi. When a disease is detected, the Arduino activates the relay connected to the fertilizer pump.

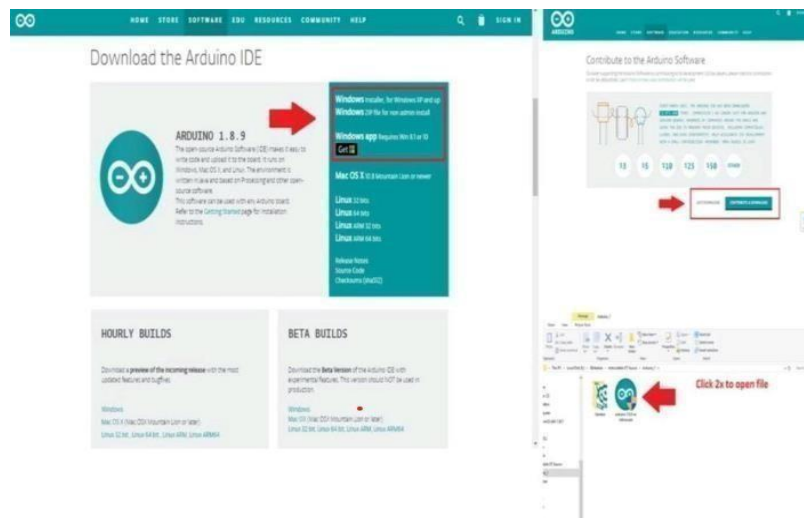
The relay acts as an electronic switch that allows the pump to operate. Once the relay is activated, the pump pushes fertilizer through the pipe, directing it toward the spray nozzle.

The nozzle sprays fertilizer directly onto the affected plant area. This targeted spraying approach ensures that fertilizer is applied only where needed, thereby reducing wastage and improving treatment efficiency.

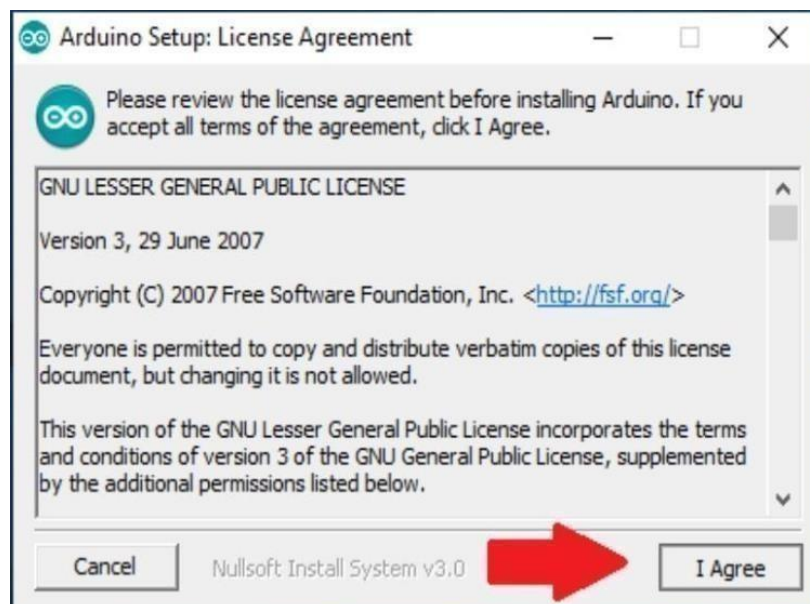
4.1.7 Installation of Arduino Mega IDE

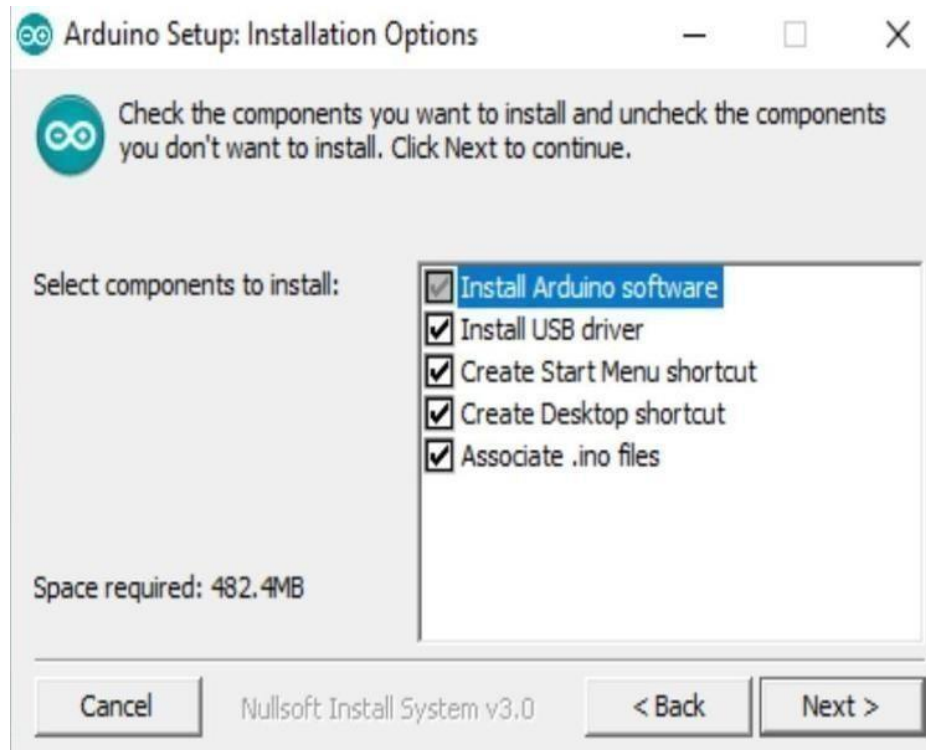
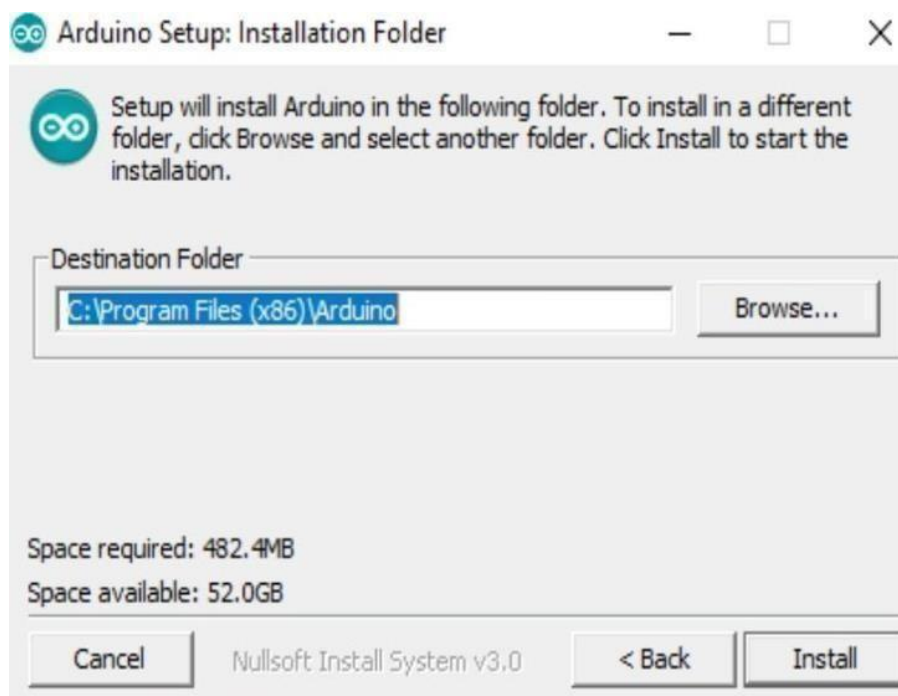
To install Arduino Mega IDE on your Windows PC, follow the next instructions:

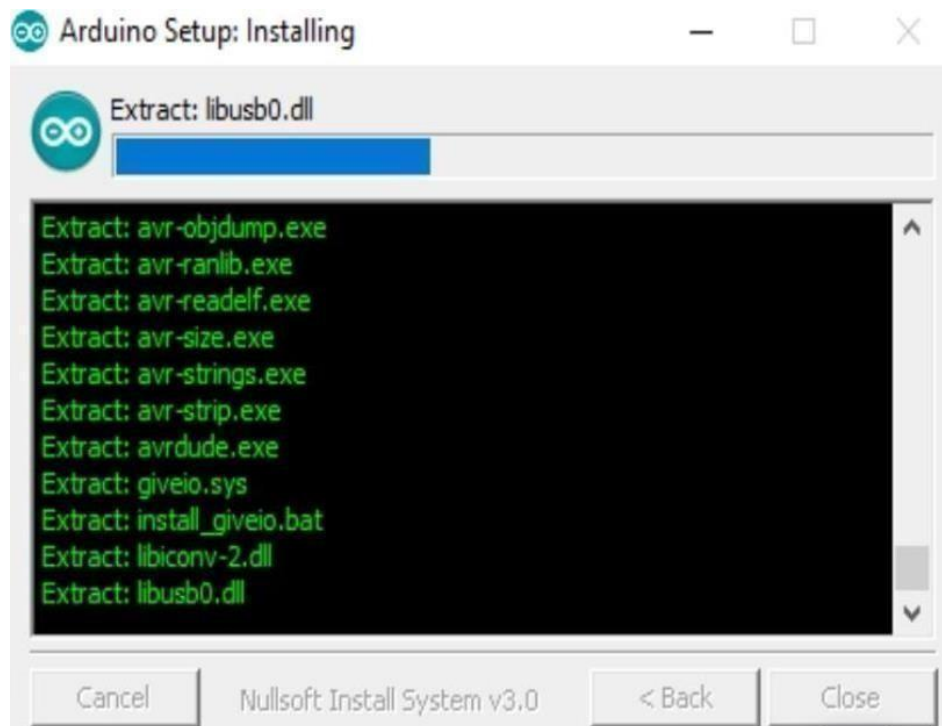
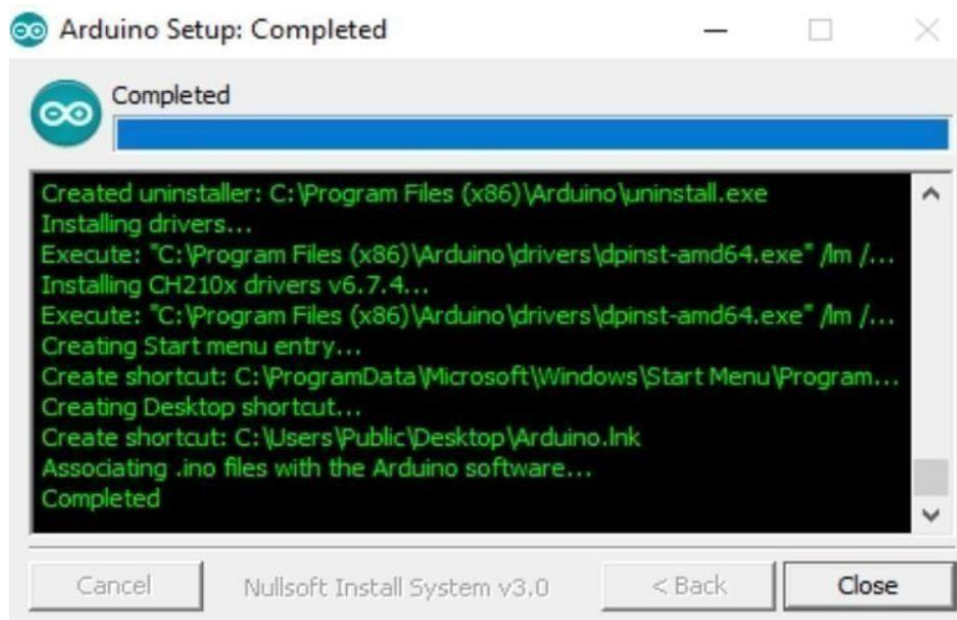
Step1: Download File Arduino IDE

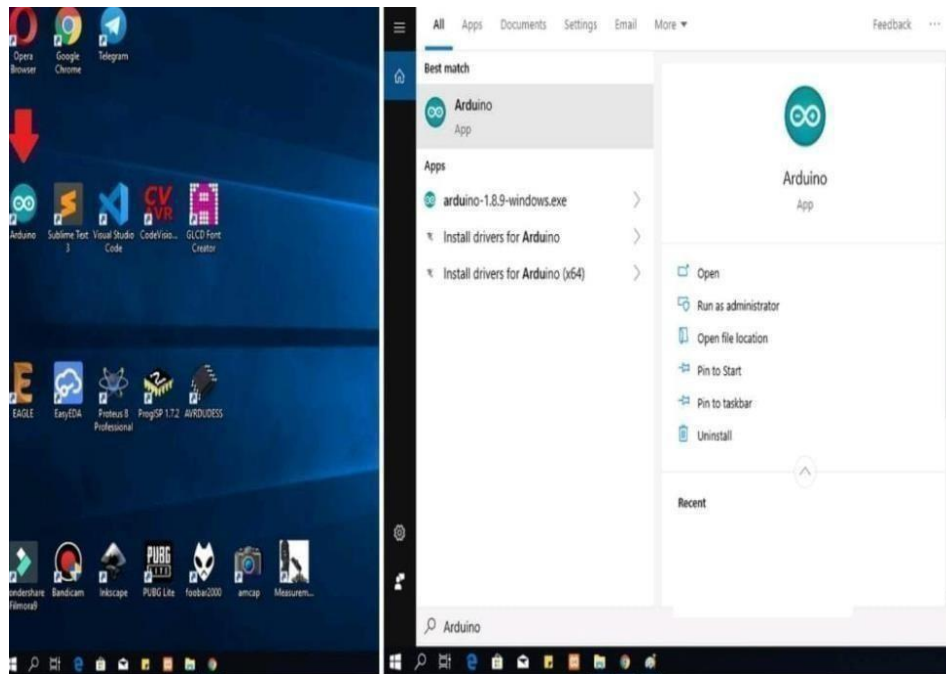


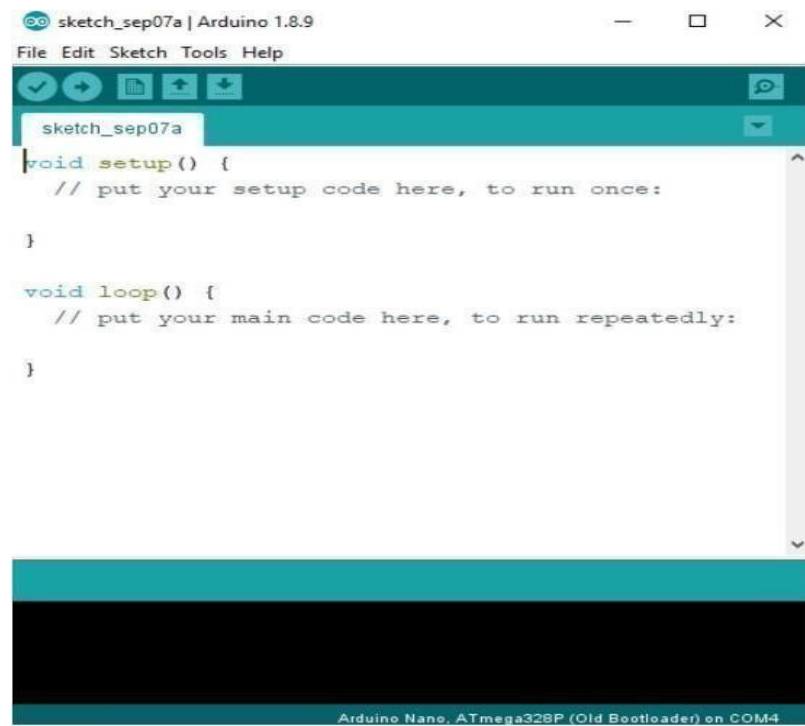
Step2: License Agreement



Step3: Installation Option**Step4: Installation Folder**

Step5: Installing Process**Step6:** Installation Complete

Step7: Open Arduino IDE**Step8: Display Arduino IDE**



4.1.8 Hardware Code

```
#include <SoftwareSerial.h>

int m1 = A0;
int m2 = A1;
int m3 = A2;
int m4 = A3;

int relay1 = 2;
int relay2 = 3;
int relay3 = 4;

SoftwareSerial mySerial(5, 6);

String value = "";
String data = "";

void setup() {
  Serial.begin(9600);
  mySerial.begin(9600);

  pinMode(m1, OUTPUT);
  pinMode(m2, OUTPUT);
  pinMode(m3, OUTPUT);
  pinMode(m4, OUTPUT);

  pinMode(relay1, OUTPUT);
  pinMode(relay2, OUTPUT);
  pinMode(relay3, OUTPUT);
  // Motors OFF
  Stop();
```

```
// ACTIVE HIGH RELAY → LOW = OFF
digitalWrite(relay1, LOW);
digitalWrite(relay2, LOW);
digitalWrite(relay3, LOW);
}

void loop() {

// ===== Serial pump control from Raspberry Pi =====
if (Serial.available() > 0) {
  data = Serial.readStringUntil('\n');
  data.trim(); // VERY IMPORTANT
  Serial.println(data);

  if (data == "high") {
    pump1();
  } else if (data == "low") {
    pump2();
  } else if (data == "medium") {
    pump3();
  }
}

// ===== Manual & Voice command =====
if (mySerial.available() > 0) {
  value = mySerial.readStringUntil('\n');
  value.trim();

  if (value == "F0") {
    forward();
  } else if (value == "B0") {
    backward();
  } else if (value == "L0") {
```

```
    left();
    delay(1000);
    Stop();
} else if (value == "R0") {
    right();
    delay(1000);
    Stop();
} else if (value == "X0") {
    Stop();
} else if (value == "T0") {
    Serial.println("ON");
}
}
}

// ===== MOTOR FUNCTIONS =====

void backward() {
    digitalWrite(m1, LOW);
    digitalWrite(m2, HIGH);
    digitalWrite(m3, HIGH);
    digitalWrite(m4, LOW);
}

void forward() {
    digitalWrite(m1, HIGH);
    digitalWrite(m2, LOW);
    digitalWrite(m3, LOW);
    digitalWrite(m4, HIGH);
}

void right() {
    digitalWrite(m1, LOW);
    digitalWrite(m2, HIGH);
```

```
digitalWrite(m3, LOW);
digitalWrite(m4, HIGH);
}

void left() {
  digitalWrite(m1, HIGH);
  digitalWrite(m2, LOW);
  digitalWrite(m3, HIGH);
  digitalWrite(m4, LOW);
}

void Stop() {
  digitalWrite(m1, LOW);
  digitalWrite(m2, LOW);
  digitalWrite(m3, LOW);
  digitalWrite(m4, LOW);
}

// ===== PUMP FUNCTIONS =====
//ACTIVE HIGH RELAY → HIGH = ON, LOW = OFF
void pump1() {
  digitalWrite(relay1, HIGH); // ON
  delay(6000);
  digitalWrite(relay1, LOW); // OFF
}

void pump2() {
  digitalWrite(relay2, HIGH);
  delay(2000);
  digitalWrite(relay2, LOW);
}

void pump3() {
  digitalWrite(relay3, HIGH);
  delay(4000);
```

4.1.9 Python and YOLOv8 Environment Setup:

Python and YOLOv8 Environment Setup

Python is the primary programming language used to implement the disease detection system in this project. The YOLOv8 deep learning model is executed using Python on the Raspberry Pi.

To set up the environment, Python and the required libraries are installed on the Raspberry Pi. The main libraries used in this project include:

- Ultralytics YOLOv8
- PyTorch
- NumPy
- OpenCV

These libraries provide the necessary tools for image processing, deep learning, and model execution.

The YOLOv8 model is trained using a dataset containing labeled images of tomato plants with different disease severity levels. The dataset is divided into training and validation sets to improve the learning process. During training, the model learns to identify infected regions in plant images.

After the training process is completed, the trained model weights are saved. The model is then deployed on the Raspberry Pi for real-time operation.

During system operation, the Python program loads the trained YOLOv8 model and analyzes images captured by the webcam connected to the Raspberry Pi. The system detects plant disease severity and produces output predictions, which are used to determine the appropriate fertilizer spraying action.

Using Python with YOLOv8 enables fast and accurate disease detection, which is essential for implementing the automated fertilizer spraying mechanism.

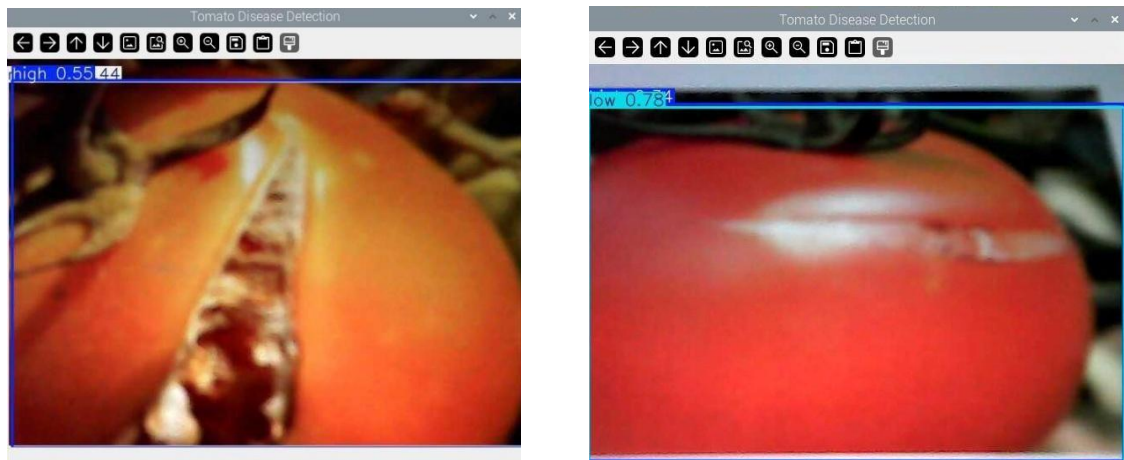


Fig.4.5: YOLOv8 based tomato disease detection output

4.1.10 OpenCV Installation and Image Processing:

OpenCV Installation and Image Processing

OpenCV (Open Source Computer Vision Library) is used in this project for image acquisition and preprocessing. It provides a wide range of functions for real-time computer vision tasks.

In the Smart Fertilizer Spraying System, OpenCV is used to capture images from the USB webcam connected to the Raspberry Pi. The library processes the captured frames before they are passed to the YOLOv8 deep learning model for analysis.

OpenCV is installed using Python package management tools such as pip. After installation, the Python program imports the OpenCV library to utilize its functions. The webcam is initialized using the VideoCapture function provided by OpenCV.

The captured frames may undergo several preprocessing steps, such as:

- Image resizing
- Normalization
- Color conversion

These preprocessing steps improve the accuracy of the deep learning model. The processed images are then provided as input to the disease detection system.

OpenCV plays an essential role in enabling real-time image capture and processing, allowing the system to continuously monitor plant health and detect disease severity.

4.1.11 Raspberry Pi Configuration:

Raspberry Pi Configuration

Raspberry Pi acts as the main processing unit of the Smart Fertilizer Spraying System. It is responsible for capturing plant images, running the disease detection model, and sending control commands to the Arduino microcontroller.

The Raspberry Pi is installed with a Linux-based operating system called Raspberry Pi OS. The operating system is written to a microSD card and inserted into the Raspberry Pi board. After booting, the Raspberry Pi is configured with the required software libraries and tools needed for the project.

Network connectivity is enabled through:

- Wi-Fi
- Ethernet

This allows remote access using tools such as RealVNC Viewer. The Raspberry Pi is also configured to communicate with external hardware devices using USB ports and GPIO pins.

The Python programming environment is installed on the Raspberry Pi, along with the required libraries:

- OpenCV
- PyTorch
- YOLOv8 framework

These libraries enable the system to process images and perform deep learning-based disease detection.

After completing the setup, the Raspberry Pi continuously runs the Python program. The program captures plant images from the webcam and analyzes them using the trained deep learning model. Based on the prediction results, the Raspberry Pi sends commands to the Arduino for fertilizer spraying.

4.1.12 System Workflow Implementation:

System Workflow Implementation

The Smart Fertilizer Spraying System operates through a sequence of steps that integrate image processing, machine learning, and embedded system control. The workflow begins when the robotic platform moves across the agricultural field.

A USB webcam mounted on the robot captures real-time images of the plants. These captured images are sent to the Raspberry Pi for analysis, where they are processed using the YOLOv8 deep learning model. The model analyzes the plant images and identifies disease symptoms present in the crops.

Based on the detected infected areas, the system determines the severity level of the plant disease. Once the severity level is identified, the Raspberry Pi sends a command signal to the Arduino Uno microcontroller through serial communication.

The Arduino receives the command and activates the appropriate relay module connected to the fertilizer pump. The fertilizer pump then sprays fertilizer through the nozzle for a specific duration depending on the detected disease severity.

Plants with higher disease severity receive a longer spraying duration, while plants with lower disease severity receive a shorter spraying duration. After spraying, the robot continues moving through the field, and the same detection and spraying process is repeated for other plants.

This automated workflow enables continuous monitoring and targeted fertilizer application, helping to improve crop health while reducing unnecessary chemical wastage.

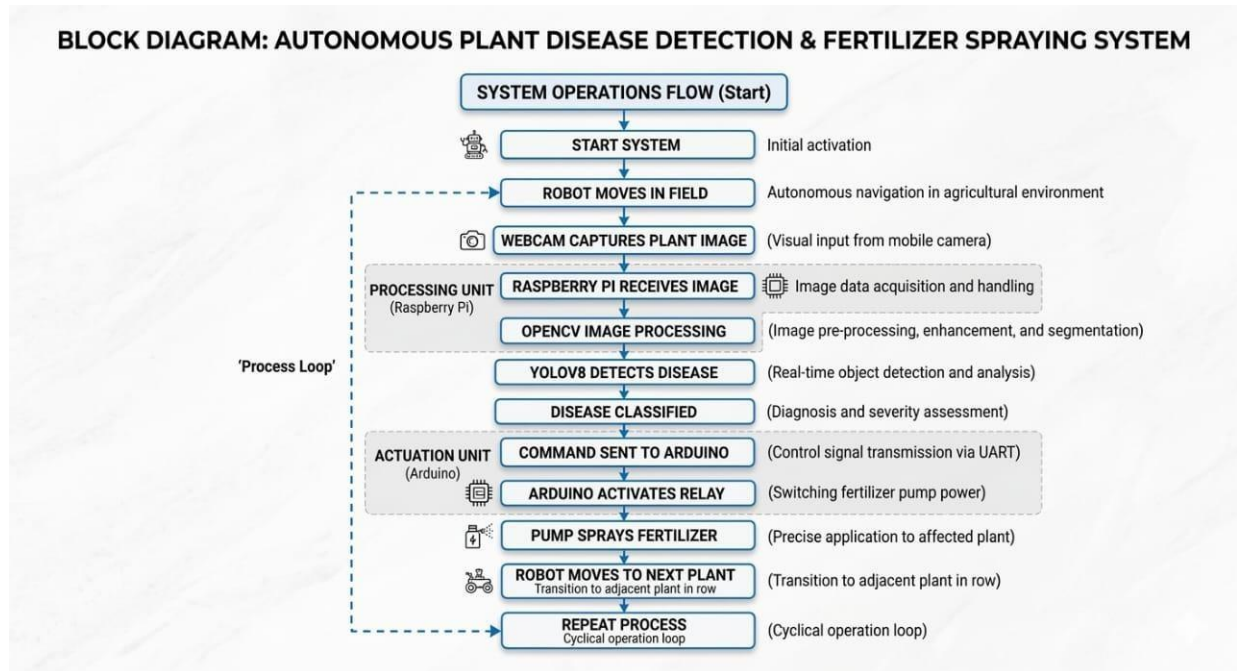


Fig.4.6: Block diagram of autonomous plant disease detection

4.1.13 Advanced IP Scanner:

Advanced IP Scanner is a network scanning tool used to detect devices connected to a local network. In this project, it is used to identify the IP address of the Raspberry Pi connected to the same Wi-Fi or LAN network.

When the Raspberry Pi is powered on and connected to the network, it automatically receives an IP address from the router. However, the user may not be aware of the assigned IP address.

Advanced IP Scanner scans the entire network and displays all connected devices along with details such as:

- IP address

- Device name
- MAC address

To use the tool, the user runs Advanced IP Scanner on a computer and performs a network scan. After scanning, the software lists all devices connected to the network, including the Raspberry Pi.

The user can identify the Raspberry Pi from the list and copy its IP address. This IP address is then used to connect to the Raspberry Pi through RealVNC Viewer.

Therefore, Advanced IP Scanner plays an important role in establishing remote access to the Raspberry Pi. It simplifies the process of locating the device on the network and ensures that the remote connection is established successfully.

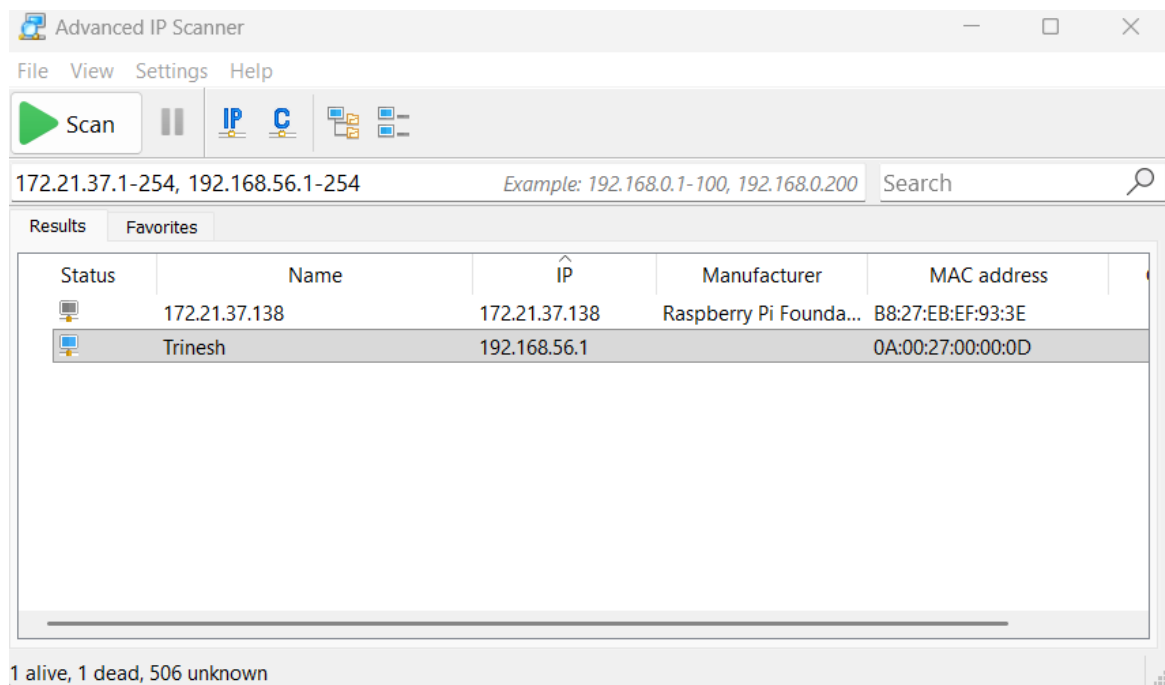


Fig.4.7: Detecting raspberry Pi IP address

4.1.14 RealVNC Viewer:

RealVNC Viewer is a remote desktop software that allows a user to access and control another computer over a network. In this project, it is used to remotely access the Raspberry Pi that controls the Smart Fertilizer Spraying System.

The Raspberry Pi is typically placed on the robot or in the agricultural field without a monitor, keyboard, or mouse. RealVNC Viewer provides a convenient way to operate the Raspberry Pi from a laptop or desktop computer.

To establish the connection, the user enters the Raspberry Pi's IP address in the RealVNC Viewer application and logs in using the appropriate username and password. The connection works using the VNC (Virtual Network Computing) protocol.

Once the connection is established, the Raspberry Pi desktop interface appears on the computer screen, allowing the user to fully control the device remotely.

Using RealVNC Viewer, the user can:

- View the Raspberry Pi desktop screen
- Run Python programs
- Monitor the image processing system
- Control the project remotely

RealVNC Viewer is also useful for:

- Monitoring the YOLOv8 model execution
- Checking camera input
- Running Python scripts
- Debugging the fertilizer spraying system

It enables the user to manage and control the Raspberry Pi efficiently without physically interacting with it.

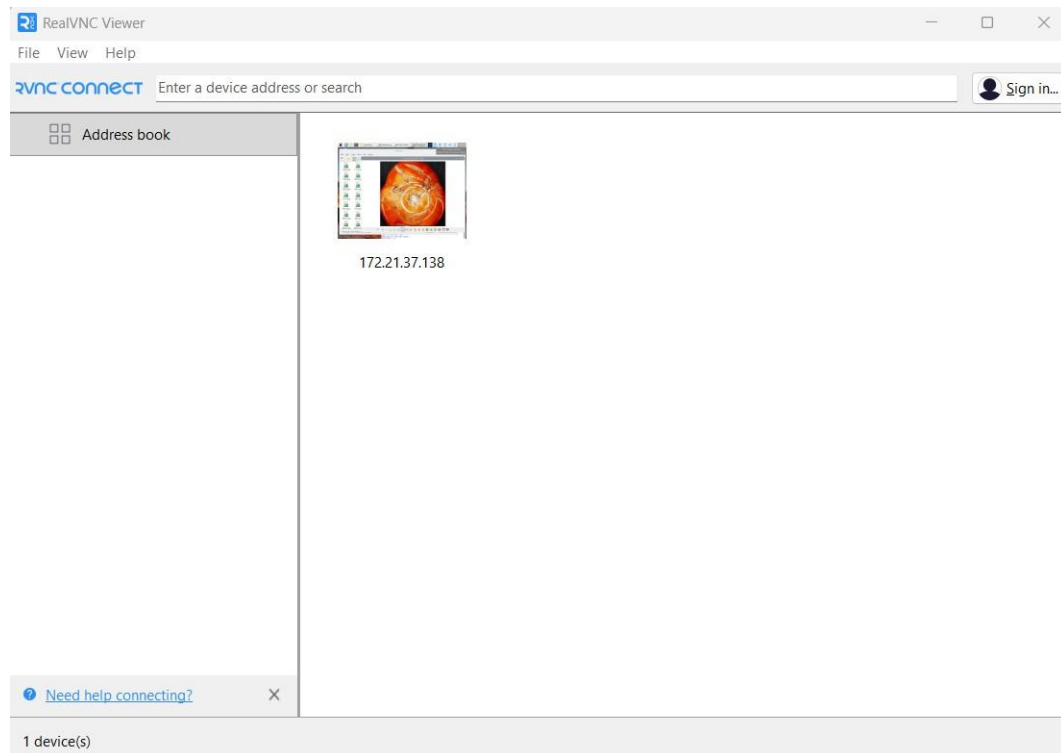


Fig.4.8: Remote access to raspberry Pi using ReanVNC Viewer

4.2 Hardware Implementation

The proposed system uses several hardware components to implement automated disease detection and fertilizer spraying.

The major hardware components include:

- Raspberry Pi
- Arduino Uno
- USB Webcam
- Motor Driver
- DC Motors
- Relay Modules
- Water Pumps
- Bluetooth Module
- Battery

- Solar Panel

These components work together to detect disease severity and control fertilizer spraying automatically.

4.2.1 Raspberry Pi

Raspberry Pi is a credit-card sized computer manufactured and designed in the United Kingdom by the Raspberry Pi foundation with the intention of teaching basic computer science to school students and every other person interested in computer hardware, programming and DIY-Do-it Yourself projects.

The Raspberry Pi is manufactured in three board configurations through licensed manufacturing deals with Newark element 14(Premier Farnell), RS Components and Egoman. These companies sell the Raspberry Pi online. Egoman produces a version for distribution solely in China and Taiwan, which can be distinguished from other Pi's by their red coloring and lack of FCC/CE marks. The hardware is the same across all manufacturers.

The Raspberry Pi has a Broadcom BCM2835 system on a chip (SoC), which includes an ARM1176JZF-S 700 MHz processor, Video Core IV GPU and was originally shipped with 256 megabytes of RAM, later upgraded (Model B & Model B+) to 512 MB. It does not include a built-in hard disk or solid-state drive, but it uses an SD card for booting and persistent storage, with the Model B+ using a MicroSD.

The Foundation provides Debian and Arch Linux ARM distributions for download. Tools are available for Python as the main programming language, with support for BBC BASIC (via the RISC OS image or the Brandy Basic clone for Linux), C, Java and Perl.



Fig.4.9: Raspberry Pi

4.2.2 Arduino Uno

Arduino Uno is a very valuable addition in the electronics that consists of USB interface, 14 digital I/O pins, 6 analog pins, and Atmega328 microcontroller. It also supports serial communication using Tx and Rx pins. There are many versions of Arduino boards introduced in the market like Arduino Uno, Arduino Due, Arduino Leonardo, Arduino Mega, however, most common versions are Arduino Uno and Arduino Mega. If you are planning to create a project relating to digital electronics, embedded system, robotics, or IoT, then using Arduino Uno would be the best, easy and most economical option. It is an open-source platform, means the boards and software are readily available and anyone can modify and optimize the boards for better functionality.

The software used for Arduino devices is called IDE (Integrated Development Environment) which is free to use and required some basic skills to learn it. It can be programmed using C and C++ language.

Some people get confused between **Microcontroller and Arduino**. While former is just an on system 40 pin chip that comes with a built-in microprocessor and later is a board that comes with the microcontroller in the base of the board, bootloader and allows easy access to input-output pins and makes uploading or burning of the program very easy.



Fig.4.10: Arduino Uno

Introduction to Arduino

- **Arduino Uno** is a microcontroller board developed by Arduino.cc which is an open-source electronics platform mainly based on AVR microcontroller Atmega328.
- First Arduino project was started in Interaction Design Institute Ivrea in 2003 by David Cuartielles and Massimo Banzi with the intention of providing a cheap and flexible way to students and professional for controlling a number of devices in the real world.
- The current version of Arduino Uno comes with USB interface, 6 analog input pins, 14 I/O digital ports that are used to connect with external electronic circuits. Out of 14 I/O ports, 6 pins can be used for PWM output.
- It allows the designers to control and sense the external electronic devices in the real world.

4.2.3 Webcam

A **webcam** is a video camera that feeds or streams an image or video in real time to or through a computer network, such as the Internet. Webcams are typically small cameras that sit on a desk, attach to a user's monitor, or are built into the hardware. Webcams can be used during a video chat session involving two or more people, with conversations that include live audio and video.

Webcam software enables users to record a video or stream the video on the Internet. As video streaming over the Internet requires much bandwidth, such streams usually use compressed formats. The maximum resolution of a webcam is also lower than most handheld video cameras, as higher resolutions would be reduced during transmission. The lower resolution enables webcams to be relatively inexpensive compared to most video cameras, but the effect is adequate for video chat sessions



Fig.4.11: Web Camera

4.2.4 Motor Driver

A motor driver is an integrated circuit chip which is usually used to control motors in autonomous robots. Motor driver act as an interface between Arduino and the motors. The most commonly used motor driver IC's are from the L293 series such as L293D, L293NE, etc. These ICs are designed to control 2 DC motors simultaneously. L293D consist of two H-bridge. H-bridge is the simplest circuit for controlling a low current rated motor. We will be referring the motor driver IC as L293D only. L293D has 16 pins.

DC Motors

A direct current (DC) motor is a type of electric machine that converts electrical energy into mechanical energy. DC motors take electrical power through direct current, and convert this energy into mechanical rotation. DC motors use magnetic fields that occur from the electrical currents generated, which powers the movement of a rotor fixed within the output shaft.

The output torque and speed depends upon both the electrical input and the design of the motor. How DC motors work The term 'DC motor' is used to refer to any rotary electrical machine that converts direct current electrical energy into mechanical energy. DC motors can vary in size and power from small motors in toys and appliances to large mechanisms that power vehicles, pull elevators and hoists, and drive steel rolling mills.

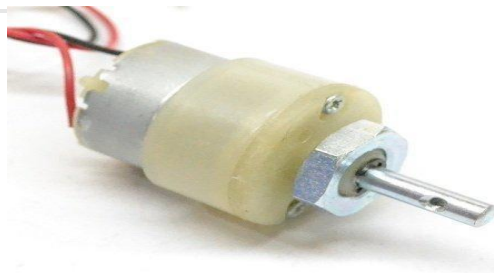


Fig.4.12: Dc Motor

4.2.5 Relay Module

A relay is an electromagnetic switch that is used to turn on and turn off a circuit by a low power signal, or where several circuits must be controlled by one signal. Most of the highend industrial application devices have relays for their effective working. Relays are simple switches which are operated both electrically and mechanically. Relays consist of an electromagnet and also a set of contacts. The switching mechanism is carried out with the help of the electromagnet. There are also other operating principles for its working. But they differ according to their applications. Most of the devices have the application of relays.

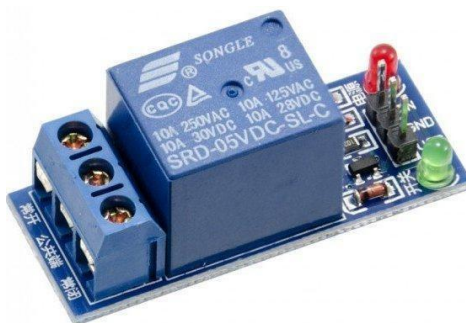


Fig.4.13: Relay Module

4.2.6 Water Pump

DC powered pumps use direct current from motor, battery, or solar power to move fluid in a variety of ways. Motorized pumps typically operate on 6, 12, 24, or 32 volts of DC power. Solar-powered DC pumps use photovoltaic (PV) panels with solar cells that produce direct current when exposed to sunlight.



Fig.4.14: DC Water Pump

4.2.7 Battery

A rechargeable battery is an energy storage device that can be charged again after being discharged by applying DC current to its terminals. Rechargeable allow for multiple usages from a cell, reducing waste and generally providing a better long-term investment in terms of dollars spent for usable device time. This is true even factoring in the higher purchase price of rechargeable and the requirement for a charger. A rechargeable battery is generally a more sensible and sustainable replacement to one-time use batteries, which generate current through a chemical reaction in which a reactive anode is consumed. The anode in a rechargeable battery gets consumed as well but at a slower rate, allowing for many charges and discharges.



Fig.4.15: Battery

4.2.8 Bluetooth module

Introduction

- It is used for many applications like wireless headset, game controllers, wireless mouse, wireless keyboard and many more consumer applications.
- It has range up to <100m which depends upon transmitter and receiver, atmosphere, geographic & urban conditions.
- It is IEEE 802.15.1 standardized protocol, through which one can build wireless Personal Area Network (PAN). It uses frequency-hopping spread spectrum (FHSS) radio technology to send data over air.
- It uses serial communication to communicate with devices. It communicates with microcontroller using serial port (USART).

HC-05 Bluetooth Module

- HC-05 is a Bluetooth module which is designed for wireless communication. This module can be used in a master or slave configuration.



Fig.4.16: HC-05 Bluetooth Module

CHAPTER 5

TESTING

5.1 Testing Approach

Testing is an essential phase in the development of any system to ensure that the implemented solution functions correctly and meets the required objectives. The Smart Fertilizer Spraying System was tested to verify the reliability of disease severity detection, communication between system components, and the correct functioning of the automated fertilizer spraying mechanism.

The testing process focused on both software testing and hardware testing. Software testing ensured that the YOLOv8 deep learning model running on the Raspberry Pi correctly detected and classified tomato disease severity levels. Hardware testing verified the interaction between the Raspberry Pi and Arduino Uno, including the control of DC motors, relay modules, and fertilizer pumps.

Several test scenarios were conducted using tomato images captured through a USB webcam. The system processed these images in real time and classified the severity of the disease into three categories: low, medium, and high. Based on the detected severity level, the system automatically activated the corresponding fertilizer spraying duration.

The testing approach ensured that the entire system—from image acquisition to hardware actuation—operated correctly under controlled experimental conditions.

5.2 Features to be tested

The following key features of the proposed system were tested to ensure correct system functionality.

1. Image Acquisition

The USB webcam connected to the Raspberry Pi was tested to verify its ability to capture real-

time images of tomato plants. The captured images were checked for clarity and proper transmission to the YOLOv8 model for processing.

2. Disease Severity Detection

The deep learning model was tested to ensure accurate classification of tomato disease severity levels. The system was verified to correctly classify input images into **low, medium, and high severity categories**.

3. Serial Communication

The communication between Raspberry Pi and Arduino Uno was tested using USB serial communication. The Raspberry Pi sends severity commands (“low”, “medium”, or “high”) to the Arduino after disease detection.

4. Relay and Pump Operation

The relay modules connected to the Arduino were tested to ensure correct activation of the fertilizer pumps. Each relay was verified to respond properly to commands received from the Raspberry Pi.

5. Fertilizer Spraying Control

The spraying mechanism was tested to verify that fertilizer spraying durations correspond to the detected severity levels:

Low severity → 2 seconds Medium severity → 4 seconds High severity → 6 seconds

6. Robot Movement

The DC motors and motor driver were tested to ensure smooth movement of the robotic platform in different directions.

7. Power Supply and Solar Support

The power supply system consisting of a 12V battery and solar panel was tested to verify continuous system operation.

5.3 Testing tools and environment

The testing of the Smart Fertilizer Spraying System was conducted using the following tools and environment.

Hardware Components

- Raspberry Pi (Main Processing Unit)
- Arduino Uno (Control Unit)
- USB Webcam
- DC Motors
- Relay Modules
- Water Pumps
- Motor Driver
- 12V Battery and Solar Panel
- Bluetooth Module

Software Tools

- Python Programming Language

- Ultralytics YOLOv8 Framework
- PyTorch Deep Learning Framework
- Arduino IDE
- OpenCV for Image Processing

Operating System

- Raspbian OS installed on Raspberry Pi

5.4 Testing Environment

The testing experiments were performed in a controlled environment using tomato images captured through a webcam. The system processed these images in real time and controlled the hardware components accordingly.

5.5 Testcases

Test cases were designed to evaluate the correct functioning of each module of the proposed system.

Test Case ID	Test Description	Input	Expected Output	Result
TC1	Image capture test	Webcam activated	Image captured successfully	Pass
TC2	Severity detection	Tomato image	Correct severity classification	Pass
TC3	Serial communication	Severity label	Command sent to Arduino	Pass
TC4	Pump activation	Low severity	Pump activated for 2 sec	Pass
TC5	Pump activation	Medium severity	Pump activated for 4 sec	Pass
TC6	Pump activation	High severity	Pump activated for 6 sec	Pass
TC7	Robot movement	Movement command	Robot moves correctly	Pass

Table5.1: Testcases

5.5.1 Non-Functional Requirements

Non-functional requirements describe the system characteristics such as performance, reliability, and usability.

Reliability: The system consistently performs disease detection and fertilizer spraying operations without system failure during testing.

Efficiency: The YOLOv8 model processes images quickly, enabling real-time detection and response.

Usability: The system requires minimal user interaction once deployed, making it suitable for practical agricultural use.

5.5.2 Portability

The system is designed to be portable and can be easily deployed in agricultural fields. The robotic platform allows the system to move across crop areas while performing disease detection and fertilizer spraying.

The use of Raspberry Pi and Arduino makes the system compact and easily transportable.

5.5.3 Speed

Speed is an important factor in real-time agricultural monitoring systems. The YOLOv8 model processes input images efficiently, allowing rapid disease detection and quick response of the spraying mechanism.

The serial communication between Raspberry Pi and Arduino ensures that commands are transmitted with minimal delay, enabling smooth system operation.

CHAPTER 6

RESULTS

The proposed Smart Fertilizer Spraying System with Plant Disease Severity Analysis was tested to evaluate its performance in detecting disease severity and controlling fertilizer spraying automatically. The system integrates a YOLOv8 deep learning model running on a Raspberry Pi with an Arduino-based hardware control mechanism.

During the experimental evaluation, tomato images were captured using a USB webcam connected to the Raspberry Pi. The trained YOLOv8 model analyzed these images and classified the disease severity into three categories: low, medium, and high. The model was able to successfully identify the severity levels based on the features learned during training.

The trained model achieved an overall classification accuracy of approximately 75–80% during testing. This indicates that the system can effectively differentiate between different disease severity levels in tomato crops. The detection results were displayed on the screen with bounding boxes and severity labels.

Once the severity level was detected, the Raspberry Pi transmitted the corresponding command to the Arduino Uno through USB serial communication. The Arduino then activated the fertilizer spraying mechanism according to the detected severity level.

The fertilizer spraying durations were predefined as follows:

Severity Level

Spraying Duration

Low

2 seconds

Medium

4 seconds

High

6 seconds

Experimental testing confirmed that the relay modules and water pumps operated correctly according to the received commands. When the system detected low severity, the pump activated for a shorter duration, while higher severity resulted in longer spraying duration.

The robotic platform was also tested to ensure proper movement and navigation. The DC motors controlled by the Arduino allowed the robot to move forward, backward, left, and right. The movement commands were successfully executed through the Bluetooth module and manual control interface.

Additionally, the solar-assisted power system provided stable power supply to the hardware components. The battery and solar panel combination ensured continuous operation of the system during testing.

Overall, the experimental results demonstrate that the proposed system successfully integrates deep learning-based disease detection with automated fertilizer spraying. The system reduces manual labor, improves fertilizer usage efficiency.

We experimentally evaluated the proposed Smart Fertilizer Spraying System to assess its effectiveness in classifying disease severity and controlling fertilizer application. The YOLOv8n model was tested using a dataset containing 150 images divided into three severity classes: low, medium, and high.

The trained model achieved an overall classification accuracy of approximately 75–80% on the test dataset. Fig. 6.1 shows the training and validation accuracy obtained during the model training process.

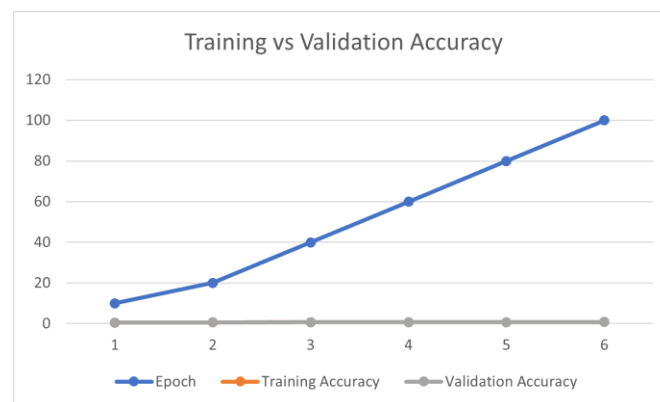


Fig. 6.1. Training and validation accuracy

To further evaluate classification performance, a confusion matrix was generated as shown in Fig. 6.2 The diagonal elements represent correct classifications.

Actual / Predicted	Low	Medium	High
Low	42	5	3
Medium	6	38	6
High	4	7	39

Fig. 6.2 Confusion matrix of YOLOv8

Precision and recall values were also analyzed to evaluate the reliability of the classification model.

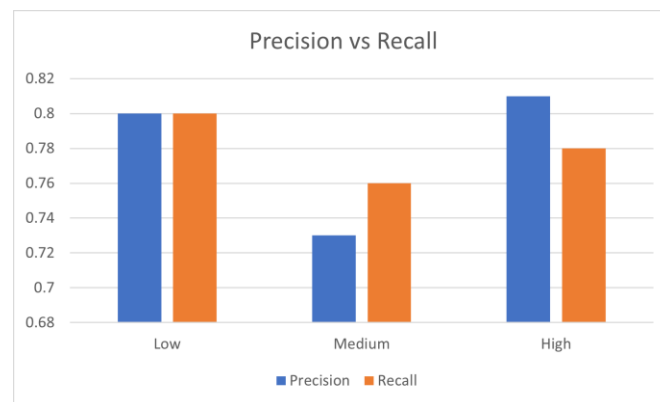


Fig. 6.3 Precision and recall comparison

During real-time deployment, the Raspberry Pi processed webcam frames and transmitted the predicted severity label (“low”, “medium”, or “high”) to the Arduino Uno via USB serial communication. The spraying duration was adjusted according to the predicted severity level.

CONCLUSION

In conclusion, the proposed Smart Fertilizer Spraying System demonstrates an effective approach for improving precision agriculture through the integration of deep learning and embedded hardware technologies. The system combines a YOLOv8n-based disease severity classification model with automated fertilizer spraying control, enabling intelligent decision-making based on the detected severity of tomato diseases. By connecting the predicted severity levels with predefined fertilizer spraying durations, the system ensures that crops receive the appropriate amount of fertilizer according to their condition. This approach reduces manual intervention while improving the efficiency and accuracy of fertilizer application in agricultural fields.

Experimental evaluations showed that the developed model achieved an overall classification accuracy of approximately 75–80 percent, indicating its capability to successfully distinguish between low, medium, and high disease severity levels in controlled environments. The reliable communication established between the Raspberry Pi and Arduino microcontroller allowed effective control of pumps and motors, ensuring synchronized robotic movement and accurate fertilizer spraying. In addition, the integration of solar power enhances the sustainability of the system by providing an alternative energy source, making the solution practical for real-world agricultural environments where continuous power supply may not always be available.

The implementation of severity-based fertilizer spraying offers significant advantages over traditional uniform spraying methods. By applying fertilizers only where necessary and in appropriate quantities, the system reduces fertilizer wastage, minimizes environmental pollution, and improves overall resource utilization. This targeted approach also contributes to improved crop health and productivity while supporting environmentally friendly farming practices. Furthermore, the combination of computer vision, deep learning, and embedded system technologies highlights the growing potential of intelligent systems in modern agriculture.

Despite its promising performance, certain limitations exist within the current system. The dataset used for training the model was relatively limited in size, which may restrict the model's ability to generalize across diverse environmental conditions. Additionally, the absence of extensive data augmentation techniques and the reliance on CPU-based training limited the optimization potential of the deep learning model. Addressing these limitations through larger datasets, improved training strategies, and enhanced computational resources can significantly improve system performance and reliability.

Overall, the proposed Smart Fertilizer Spraying System demonstrates that integrating artificial intelligence with automated agricultural equipment can play an important role in transforming traditional farming practices. The system represents a practical step toward intelligent and sustainable agriculture by enabling data-driven decision-making and efficient fertilizer management. Such technology-driven approaches have the potential to significantly improve crop productivity while promoting environmentally responsible farming methods.

REFERENCES

- [1] P. M. Manikar, S. Ghorpade, and M. Adawadkar, "Plant Leaf Disease Detection and Classification Using Image Processing Techniques," *International Engineering Journal*, vol. 2, no. 4, pp. 123-128, 2015.
- [2] B. Patil, H. Panchal, S. Yadav, A. Singh, and D. Patil, "Plant Monitoring Using Image Processing, Raspberry Pi and IoT," *Journal of Engineering and Technology*, vol. 4, no. 10, pp. 456-462, 2017.
- [3] N. V. Dumare and S. S. Mungona, "Identification of Cotton Leaf Diseases Using Raspberry Pi," *International Journal on Recent and Technology Trends in Computing and Communication*, vol. 5, no. 5, pp. 212-217, 2017.
- [4] P. K. Kharde and H. H. Kulkarni, "A Special Method for the Detection of Grape Leaf Diseases," *International Journal of Scientific Research*, vol. 2, no. 4, pp. 89-93, 2016.
- [5] J. Sethupathy and Veni S., "OpenCV Based Disease Detection of Mango Leaves," *International Journal of Engineering and Technology*, vol. 8, no. 5, pp. 98-102, 2016.
- [6] S. Mohanty, D. P. Hughes, and M. Salath'e, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, 2016.
- [7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779-788.
- [8] W. Liu et al., "SSD: Single Shot MultiBox Detector," in *Proc. European Conference on Computer Vision*, 2016, pp. 21-37.
- [9] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Proc. Advances in Neural Information Processing Systems*, 2012, pp. 1097-1105.
- [10] B. Yang et al., "Development of a Smart Pesticide Spraying Robot Using Image Processing and Machine Learning," *IEEE Access*, vol. 8, pp. 87234-87246, 2020.
- [11] R. Li, M. Wang, Y. Zhang, and Q. Sun, "A Survey of Deep Learning for Crop Disease Detection and Diagnosis," *IEEE Access*, vol. 9, pp. 36622-36641, 2021.
- [12] D. Zhang, X. Li, and Y. Wang, "Precision Agriculture: A Review of Machine Learning Applications," *IEEE Access*, vol. 10, pp. 12345-12357, 2022.

PATENT PUBLICATION STATUS

Dear Sir/Madam, we have successfully filed our Indian publication patent; a minimum of 1 week and a maximum of 45 days later, the patent will be published.

Once published you

Link:

<https://iprsearch.ipindia.gov.in/PublicSearch/PublicationSearch/ApplicationStatus>

Application Number:

202641030974

Application Details	
APPLICATION NUMBER	202641030974
APPLICATION TYPE	ORDINARY APPLICATION
DATE OF FILING	14/03/2026
APPLICANT NAME	Srinivasa Ramanujan Institute of Technology
TITLE OF INVENTION	IoT-Based Fertilizer Spraying System with Plant Disease Severity Analysis
FIELD OF INVENTION	MECHANICAL ENGINEERING
E-MAIL (As Per Record)	mail2patentipr@gmail.com
ADDITIONAL-EMAIL (As Per Record)	
E-MAIL (UPDATED Online)	
PRIORITY DATE	
REQUEST FOR EXAMINATION DATE	--
PUBLICATION DATE (U/S 11A)	27/03/2026

Application Status	
APPLICATION STATUS	Awaiting Request for Examination
View Documents	

→
Filed
→
Published
→
RQ Filed
→
Under Examination
→
Disposed

In case of any discrepancy in status, kindly contact ipo-helpdesk@nic.in

INTERSHIP CERTIFICATES



शिक्षा मंत्रालय
MINISTRY OF
EDUCATION



अखिल भारतीय तकनीकी शिक्षा परिषद्
All India Council for Technical Education



NATIONAL
INTERNSHIP
PORTAL



EduSkills®
Nation Building Through Skills



Certificate of Virtual Internship

This is to certify that

REVATHI BOMMA

Srinivasa Ramanujan Institute of Technology

has successfully completed the 10-week

Ethical Hacking Virtual Internship (English Language)

During October - December 2025

May this Internship learning propel you toward a bright and successful career.

Supported by






Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education



Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills





Certificate ID :bb2137ff9f7393d031ad87962b13acea
Student ID :STU641c6b06026661679584006

GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30







शिक्षा मंत्रालय
MINISTRY OF
EDUCATION



अखिल भारतीय तकनीकी शिक्षा परिषद्
All India Council for Technical Education



EduSkills®
Nation Building Through Skills



Certificate of Virtual Internship

Y Trinesh Reddy

Srinivasa Ramanujan Institute of Technology

has successfully completed the 10-weeks

Cloud Virtual Internship

During January - March 2026

May this Internship learning propel you toward a bright and successful career.

Curriculum Provided by:



Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education



Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 478485e6dc7abf570fcc

Student ID: STU6589304f394e81703489615



GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

PROJECT CO-OP MAPPING

PROJECT TITLE :

SMART FERTILIZER SPRAYING SYSTEM USING PLANT DISEASE SEVERITY ANALYSIS

Course Outcomes	Program Outcomes											
	1	2	3	4	5	6	7	8	9	10	11	12
Understand driver safety challenges using IoT, computer vision, and sensor technologies.	X	X									X	
Analyze driver drowsiness and alcohol detectines techniques using CNN, EAR, and MQ-3 sensors.	X	X	X									X
Design and develop an intelligent system integrating Raspberry Pi, sensors, and camera modules.			X	X	X	X						
Implement real-time monitoring and alert mechanisms for unsafe driving conditions.			X	X	X	X						
Evaluate system performance using experimental analysis and testing in real-world scenarios.	X	X	X	X	X	X						
Demonstrate teamwork, communication, and ethical responsibility in developing safety-critical systems.							X	X	X	X	X	X

DETAILS OF PROJECT TEAM

	Dr. C. Sasikla M.Tech.,Ph.D. Associate Professor Computer Science and Engineering Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh Sasikala.cse@srit.ac.in
	BOMMA REVATHI 224G1A3380 Computer Science and Engineering (AI&ML) Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 224g1a3380@srit.ac.in
	GANNE SREEJA 224G1A33A9 Computer Science and Engineering (AI&ML) Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 224g1a33a9@srit.ac.in
	Y SRAVANI 224G1A33A4 Computer Science and Engineering (AI&ML) Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 224g1a33a4@srit.ac.in
	Y TRINESH REDDY 224G1A33A4 Computer Science and Engineering (AI&ML) Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 234g5a3311@srit.ac.in